



廣東工業大學

本科毕业设计（论文）

融合改进 A*算法与自适应遗传算法的移动 机器人路径规划

学 院： 自动化学院
专 业： 物联网工程
年级班别： 2022 级(1)班
学 号： 3122001342
学生姓名： 陈楚楠
指导教师： 黄旭民

2026 年 6 月

摘要

移动机器人路径规划是机器人学与人工智能领域的核心研究课题，其性能优劣直接影响机器人在仓储配送、医疗辅助、工业巡检及自动驾驶等场景中的任务执行效率、能量消耗与运行安全性。随着智能制造与智慧物流技术的快速发展，大规模复杂环境下的路径规划能力面临更高要求。在传统算法层面，A*算法与Dijkstra算法等确定性搜索方法虽能保证路径最优性，但存在搜索效率瓶颈、启发式权重固定导致的环境适应性不足，以及栅格离散化引发的路径冗余转折等问题；快速随机树（RRT）等基于采样的方法虽适用于高维空间，但路径质量波动较大且难以保证最优性；人工势场法（APF）实时性较好，却极易陷入局部极小值。在智能优化算法层面，遗传算法（GA）、粒子群优化（PSO）与蚁群算法（ACO）虽具备全局搜索能力，但分别存在收敛缓慢、易陷入局部最优、关键参数依赖人工经验等缺陷，难以满足对实时响应与解集质量均有较高要求的路径规划任务。近年来，算法融合成为路径规划领域的重要发展方向，然而现有融合方案在初始路径质量引导、遗传进化参数自适应调整及不可行路径修复等关键环节仍存在不足，制约了融合算法的整体性能与实用价值。

针对上述问题，本文提出一种融合改进A算法与自适应遗传算法的移动机器人路径规划方法。首先，在A算法中引入基于障碍物密度的动态权重因子 $\alpha(\rho)$ ，建立环境复杂度与启发函数权重的自适应映射关系，使算法在障碍物稀疏区域保持路径最优性，在障碍物密集区域增强搜索导向性以减少无效节点扩展，从而提升搜索效率；同时采用冗余点剔除与折线段简化等后处理操作，改善初始路径质量。其次，在遗传算法中设计多项自适应优化策略：利用不同权重参数组合下的改进A算法生成多条差异化可行路径作为初始种群，以缓解传统遗传算法的“冷启动”问题；构建综合考虑路径长度、平滑度、转折点数量及碰撞惩罚的多目标适应度函数，引导种群向兼顾经济性与行驶平稳性的方向进化；制定基于个体适应度值的自适应交叉概率与变异概率调节策略，对优良个体降低算子作用概率以保护优质基因片段，对劣质个体提高算子作用概率以促进探索与改良，并引入精英保留机制与停滞破除机制以防止早熟收敛。最后，针对遗传进化过程中交叉变异操作产生的不可行路径，设计以“A算法重连接为主、备选点位搜索为辅”的两层级递

归式路径修复策略，有效保障输出路径的连通性与可行性，提高种群利用率。

实验在 1500×1500 规模的真实栅格地图上分五个阶段进行系统验证，并与标准 A、Dijkstra、JPS 及 JPS+GA 等基准算法进行全面对比。结果表明：改进 A 算法的搜索效率较标准 A 提升 19.6%至 75.1%，且随规划里程增加优势更为显著；融合方案在路径总长度指标上较纯改进 A 缩短 3.9%至 8.5%，转折点数量减少 48%至 92%；在路径平滑度指标上较 JPS 优化 7.9 倍至 8.5 倍，表现出显著的路径品质优势；在运算效率上较 Dijkstra 算法提升 64%至 98%。与 JPS+GA 方案相比，Improved A*+GA 在平滑度上优化 8.1 倍至 8.7 倍，综合性能均衡性更为突出。所有生成路径均通过有效性验证，证明了本文方法在复杂栅格环境中具有良好的可靠性与实用价值。

关键词：机器人，路径规划，A*算法，自适应遗传算法，启发函数，动态权重

Abstract

Mobile robot path planning is a core research topic in robotics and artificial intelligence. Its performance directly affects the robot's task execution efficiency, energy consumption, and operational safety in scenarios such as warehousing and distribution, medical assistance, industrial inspection, and autonomous driving. With the rapid development of intelligent manufacturing and smart logistics technologies, path planning capabilities in large-scale complex environments face higher demands. At the traditional algorithm level, deterministic search methods such as A* and Dijkstra's algorithm can guarantee path optimality, but they suffer from bottlenecks in search efficiency, insufficient environmental adaptability due to fixed heuristic weights, and path redundancy caused by grid discretization. Sampling-based methods such as Rapid Random Tree (RRT) are suitable for high-dimensional spaces, but path quality fluctuates greatly and it is difficult to guarantee optimality. Artificial Potential Field (APF) has good real-time performance but is prone to getting trapped in local minima. At the intelligent optimization algorithm level, Genetic Algorithm (GA), Particle Swarm Optimization (PSO), and Ant Colony Optimization (ACO) have global search capabilities, but they suffer from slow convergence, susceptibility to local optima, and reliance on human experience for key parameters, making it difficult to meet the requirements of path planning tasks with high demands for real-time response and solution quality. In recent years, algorithm fusion has become an important development direction in the field of path planning. However, existing fusion schemes still have shortcomings in key aspects such as initial path quality guidance, adaptive adjustment of genetic evolution parameters, and repair of infeasible paths, which restricts the overall performance and practical value of fusion algorithms.

To address these issues, this paper proposes a mobile robot path planning method that integrates an improved A algorithm and an adaptive genetic algorithm. First, a dynamic weight factor $\alpha(\rho)$ based on obstacle density is introduced into the A algorithm to establish an adaptive mapping relationship between environmental complexity and heuristic function weights. This allows the algorithm to maintain path optimality in sparse obstacle regions and enhance search guidance in dense obstacle

regions to reduce the expansion of invalid nodes, thereby improving search efficiency. Simultaneously, post-processing operations such as redundant point removal and polyline segment simplification are employed to improve the initial path quality. Secondly, several adaptive optimization strategies are designed in the genetic algorithm: An improved A-algorithm with different weight parameter combinations generates multiple differentiated feasible paths as the initial population to alleviate the "cold start" problem of traditional genetic algorithms; a multi-objective fitness function considering path length, smoothness, number of turning points, and collision penalties is constructed to guide the population to evolve in a direction that balances economy and driving stability; an adaptive crossover and mutation probability adjustment strategy based on individual fitness values is formulated, reducing the operator action probability for superior individuals to protect high-quality gene fragments, and increasing the operator action probability for inferior individuals to promote exploration and improvement; an elite preservation mechanism and a stagnation elimination mechanism are introduced to prevent premature convergence. Finally, for infeasible paths generated by crossover and mutation operations during genetic evolution, a two-level recursive path repair strategy is designed, primarily using A-algorithm reconnection and secondarily using alternative point search, effectively ensuring the connectivity and feasibility of the output path and improving population utilization.

The experiment was conducted in five stages on a real 1500×1500 grid map for system validation, and a comprehensive comparison was made with benchmark algorithms such as Standard A, Dijkstra, JPS, and JPS+GA. The results show that the improved A algorithm improves search efficiency by 19.6% to 75.1% compared to Standard A, with the advantage becoming more significant as the planned mileage increases. The fusion scheme shortens the total path length by 3.9% to 8.5% compared to pure Improved A, and reduces the number of turning points by 48% to 92%. It also improves path smoothness by 7.9 to 8.5 times compared to JPS, demonstrating a significant advantage in path quality. Furthermore, it improves computational efficiency by 64% to 98% compared to the Dijkstra algorithm. Compared to the JPS+GA scheme, Improved A*+GA improves smoothness by 8.1 to 8.7 times, exhibiting a more balanced overall performance. All generated paths passed validity verification, demonstrating the

good reliability and practical value of the proposed method in complex grid environments.

Key words: Robot, path planning, A* algorithm, adaptive genetic algorithm, heuristic function, dynamic weight

目录

1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 传统路径规划算法研究	2
1.2.2 智能优化算法在路径规划中的应用	3
1.2.3 融合算法研究现状	3
1.3 本章小结	4
2 环境建模与算法基础原理	6
2.1 栅格地图构建与坐标表示	6
2.1.1 栅格地图数学模型	6
2.1.2 坐标系与邻接关系	6
2.1.3 障碍物密度计算	7
2.2 传统 A* 算法原理与局限性	7
2.2.1 A* 算法基本原理	7
2.2.2 启发式函数的选择	8
2.2.3 传统 A* 算法的局限性	8
2.3 传统遗传算法原理与局限性	9
2.3.1 遗传算法基本原理	9
2.3.2 关键算子设计	10
2.3.3 传统遗传算法的局限性	10
2.4 本章小结	11
3 融合路径规划算法设计与实现	12
3.1 融合算法总体架构	12
3.2 优化启发函数：动态权重设计	14
3.2.1 动态权重函数设计	14
3.2.2 障碍物密度计算	15
3.2.3 曼哈顿距离启发式	15
3.3 搜索策略改进	16
3.3.1 改进 A* 算法流程	16
3.3.2 8 邻域扩展策略	16
3.4 路径平滑优化	17
3.4.1 冗余点删除	17
3.4.2 折线段简化	17
3.5 本章小结	18
4 自适应遗传算法优化	19
4.1 自适应遗传算法设计	19
4.1.1 种群初始化策略	19
4.1.2 适应度函数设计	20
4.1.3 自适应交叉率	21
4.1.4 自适应变异率	21
4.1.5 精英保留策略	22

4.1.6 选择算子	22
4.2 融合架构设计	22
4.2.1 总体流程	22
4.2.2 融合优势分析	23
4.3 路径修复机制	23
4.3.1 非法点检测	24
4.3.2 A*重连策略（优先）	24
4.3.3 替代点搜索（备选）	24
4.3.4 递归修复与终止	25
4.3.5 修复效果评估	25
4.4 本章小结	25
5 仿真实验与结果分析	26
5.1 实验环境与参数设置	26
5.1.1 实验平台	26
5.1.2 算法参数设置	26
5.1.3 测试用例设计	26
5.2 评价指标体系	27
5.3 阶段一：基础路径算法对比实验	27
5.4 阶段二：遗传算法融合验证实验	32
5.5 阶段三：融合方案与多算法对比实验	35
5.6 阶段四：JPS 与融合方案对比实验	38
5.7 阶段五：两种融合方案对比实验	43
5.8 本章小结	47
6 总结与展望	48
6.1 研究工作总结	48
6.2 不足与展望	49
6.3 本章小结	49
参考文献	50
致谢	55
附录 A 核心算法源代码	56
附录 B 实验配置与参数汇总	59

1 绪论

1.1 研究背景与意义

近些年来，伴随着智能制造、智慧物流还有服务机器人等各项技术的不断往前推进，移动机器人已经相当广泛地被投用到了仓储配送、医疗辅助以及工业巡检等许许多多的行业领域当中。在这些个实际的应用场景里面头，路径规划可以说是移动机器人系统当中的一个核心环节了，它规划出来的质量是好是坏，会直接牵扯到任务执行的效率到底高不高、能量消耗的水平到底大不大，还有运行过程当中安不安全这类的关键性能指标上去^{[12][43][39]}。究竟要怎么在结构错综复杂并且还存在着好多种约束条件的作业环境里面头，规划出一条既具备高效性又带有平滑特性的行进路径来，一直以来都是这个领域当中急需去往深里头开展研究的一个关键问题^{[30][2]}。

当前对于路径规划算法的划分大体上能归入两个类别：一个类别是由 A*算法与 Dijkstra 算法担当代表的确定性搜索手段，另一个类别则是由遗传算法（GA）充当代表的智能优化途径。前一个类别在理论上是有能力确保寻获到最优解或者起码是可通行的路径的，但是在工程落地的过程中却暴露出了若干项欠缺之处。单拿 A*算法来当作例子说明的话，碰到障碍物分布比较稠密的环境状况时，这套算法就必须挨个儿去扩展数目庞大的候选节点，计算方面的开销会随着地图尺寸的变大而呈现出急剧攀升的势头；其启发函数所带有的权重数值是固定不变的那种，做不到根据局部环境复杂程度的起伏去做出带有适应性质的改动；此外，受到栅格离散化的处理方式以及搜索策略自身特性的掣肘，所产出的路径当中常常裹挟着许多冗余的转折环节，这对于机器人实际执行动作而言是很不利的。遗传算法虽说在全局寻优这一块儿握有较强的鲁棒特性，然而它的标准形态同样要去面对收敛步伐偏于缓慢、容易陷进局部最优的泥淖以及关键参数的设置对人工经验有着高度依赖这一类麻烦，要想在搜索效率同解集质量之间谋求到一个让人感到称心的平衡状态是颇具难度的^{[11][45][46]}。

上面所做的这些剖析已然点明了，单一类型的算法模式再也难以同时照顾到搜索效率、路径品质还有环境自适应能力这多重维度的诉求了。有鉴于此，本文拿出了这样一套把改进后的 A*算法同自适应遗传算法揉合到一块儿的路径规划手

法。这套手法的核心构思脉络是落在下述几个方面上的：第一个方面是把基于障碍物密度的动态权重因子植入到了 A*算法的内部，让它得以顺着环境复杂程度的线索来自动调整启发式的权重配比，以便在搜索效率同路径最优属性之间谋求一个两全其美的局面；第二个方面是把经由改进 A*算法产出过的高质量路径拿来充作遗传算法最开始的种群，再凭借自适应交叉与变异这两项策略去到全局的解空间里头进一步去优化路径总长度、平滑度等诸多维度的性能指标；第三个方面则是搭配上了一套专门构思出来的路径修补策略去确保最终输出路径在连通性与可行性这两个方面是达标过关的。本项研究的意图是在于给移动机器人在复杂栅格环境之中的自主导航行当带去一套更为高效、更加靠得住的规划方案，因而它本身就涵括了一定的理论层面的含义以及应用场景里的价值。

1.2 国内外研究现状

1.2.1 传统路径规划算法研究

A*算法由 Hart 等于 1968 年首次提出^[23]，是启发式图搜索领域的经典方法。该算法通过评估函数

$$f(n) = g(n) + h(n) \quad (1.1)$$

选择待扩展节点，其中 $g(n)$ 表示起点至当前节点 n 的已知累积代价， $h(n)$ 为节点 n 至目标点的启发式估计代价。当启发函数满足可容许性条件时，A*算法能够保证找到代价最小的路径^{[12][13][32]}。然而，经典 A*算法采用固定权重的启发函数，当环境中障碍物分布密集时，算法需扩展大量节点以绕过死胡同和凹形障碍区域，搜索效率随之显著下降。针对这一问题，国内外学者提出了多种改进方案，包括动态权重调整^[33]、多目标优化^[34]、以及结合拓扑地图的加速策略^[50]等。

Dijkstra 算法是另一种经典的最短路径求解方法，由 Dijkstra 于 1959 年提出^[24]。该算法以均匀代价搜索策略遍历图中节点，能够精确求得任意两点间的最短路径；但由于不引入任何启发式信息，其搜索范围通常覆盖大片区域，在大规模栅格地图中计算开销显著^{[12][41]}。

Jump Point Search (JPS) 算法由 Harabor 和 Grastien 于 2011 年提出^[25]，是一种面向均匀代价网格的加速搜索技术。JPS 利用路径对称性实施节点剪枝，仅对关键的“跳点”进行扩展，在规则网格地图上可将搜索效率提升至传统 A*算法的数倍

乃至数十倍。然而，JPS 的优化目标仅限于路径长度，未涉及平滑度等其他质量维度，且其跳跃式的路径点分布往往导致路径转折剧烈^{[12][39]}。

1.2.2 智能优化算法在路径规划中的应用

遗传算法（Genetic Algorithm, GA）这一概念经由 Holland 在 1975 年作出了系统性的阐述^[26]，它的本质是仿照自然界里的自然选择过程与遗传机理而构筑起来的一种全局性随机寻优手段。GA 会借助选择、交叉还有变异这几类遗传算子去往解空间当中推行并行的搜寻作业，在处理非线性问题、多峰问题以及组合优化问题的时候展露出了比较强悍的鲁棒特性^{[2][45]}。最近一些年来，GA 在路径规划这片领域内获得了相当普遍的运用，其所触及的研究方向涵盖了多目标路径优化任务、处在动态变动环境底下的路径重新规划工作等等^{[9][40]}。

只不过，一旦将标准形态的 GA 挪用到路径规划问题上去，便有几项与生俱来的毛病会浮现出来：第一个毛病在于，初始种群的形成往往仰仗随机生成的方式，缺少了从问题本身结构那里传递过来的先验引导信息，这么一来就拖累了算法在早期阶段的搜索效能；第二个毛病体现在，交叉发生概率与变异发生概率在整个进化历程当中是纹丝不动的，这就没有办法针对质量参差不齐的个体去施加有差别的操作手段——那些适应度数值偏高的个体所保有的优质基因片段相当容易在交叉过程里头遭到肢解破坏，而那些适应度数值偏低的个体呢，又很难争取到足够力度的变异动作来实现自身的改观；第三个毛病在于，伴随着进化步骤的持续向前推进，种群当中所含有的多样性成分会一点一点地流失掉，算法也就很容易栽进局部最优解的那个陷阱里头去（这也就是平常所说的早熟收敛现象）；第四个毛病则是，路径规划里头的障碍物约束应当归属为一种硬性约束条件，而标准形态的 GA 对于交叉变异操作所捣鼓出来的那些压根儿就走不通的路径，手里头是缺乏一套行之有效的处置路数的。冲着上面讲到的这一连串症结，海内外的研究人士先后拿出来了自适应参数调整的机制^{[4][18][46]}、精英留存策略^{[5][48]}、停滞局面打破的机制^{[47][49]}以及样式繁多的混合算法架构^[14]等等一系列改进性质的方案。

1.2.3 融合算法研究现状

把不同类型算法身上各自的长处捏合到一处，这条路线已经渐渐演变成了路径规划范畴里头一个备受瞩目的探究走向。眼下已有的融合招数大体上能归结到三

个门类底下去：第一个门类走的是串行级联的套路，也就是先借助计算效率偏高一些的确定性搜索算法去产出一条起始可行的路径，随后再动用智能优化算法对这条路径展开第二轮的雕琢优化；第二个门类走的是约束引导的路线，把某一种算法的输出结果拿过来当成另一种算法起步阶段的初始条件抑或是搜寻过程里的约束边界；第三个门类则走的是并行协作的途径，让好几种算法在同一时段运转起来并且相互之间可以共享搜寻所得的信息。已经有相当数量的研究结果在表明，经过了妥善谋划的融合方案是有本事在搜索效率同路径品质这两者之间谋得一个较为妥帖的折中局面的，放在复杂环境当中这种长处还会变得格外扎眼^{[1][6][35][36]}。

针对 A*算法与遗传算法的融合改进，即便当下已经积累了一定的探索经验，但在具体的实现方案里依然能发现几处不完善的地方；第一个方面是，多数研究大多只是把标准形态的 A*算法套用到起始路径的生成中，并没能把环境特征所携带的信息给充分地调动起来，也就无法把初始解的质量给提上去，导致后续的优化余地被挤占了不少；第二个方面表现在，遗传算法对于交叉和变异概率的自适应调节做得还不够细致，没能按照个体适应度的高低去执行区分化的控制，这让种群的进化效率给慢了下来；第三个方面则是针对遗传操作里出现的无效路径，现在还缺少一套逻辑严密的修补方案，往往只是把这些路径简单地扔掉了，这既把运算资源给浪费了，也把有效种群的整体规模给变薄了^{[8][15][37][38]}。

1.3 本章小结

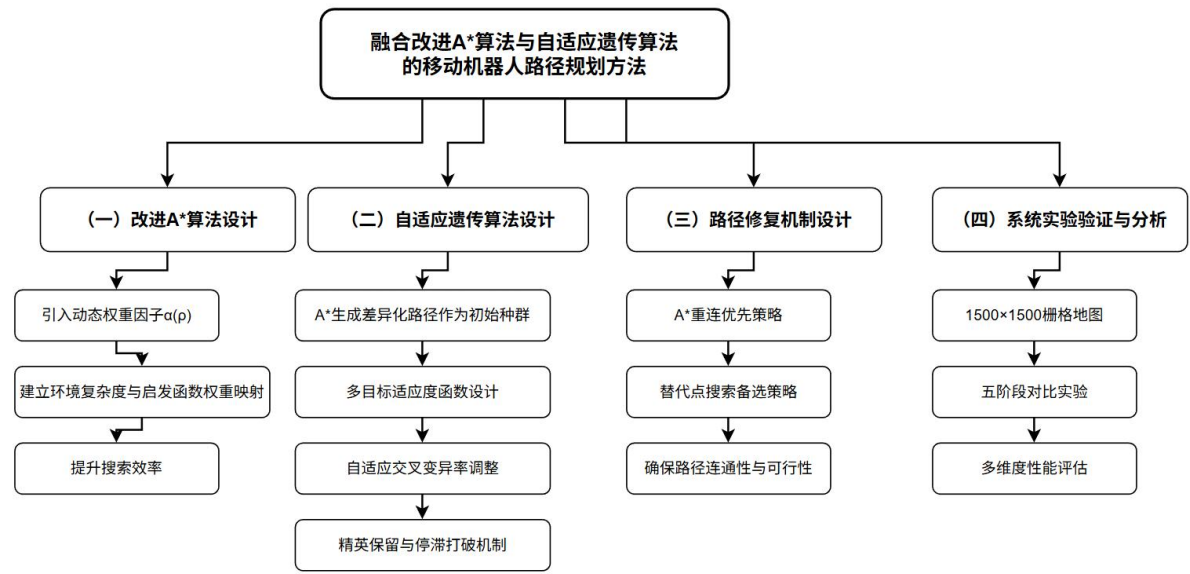


图 1.1 本文主要内容流程图

针对上述问题，本文围绕融合改进 A*算法与自适应遗传算法的移动机器人路径规划方法展开研究，主要工作包括以下四个方面：

（1）改进 A*算法设计。引入基于障碍物密度的动态权重因子 $\alpha(\rho)$ ，建立环境复杂度与启发函数权重之间的映射关系，使算法在低密度区域维持路径最优性，在高密度区域增强搜索引导力度，从而有效提升搜索效率。

（2）自适应遗传算法设计。利用不同动态权重参数的改进 A*生成多条差异化路径构成初始种群，提高初始解的质量与多样性；设计综合考虑路径长度、平滑度、转折点数量和碰撞惩罚的多目标适应度函数；提出基于个体适应度的自适应交叉率与变异率调整策略，并引入精英保留机制和停滞打破机制以防止早熟收敛。

（3）路径修复机制设计。针对遗传操作产生的不可行路径，提出“A*重连优先、替代点搜索备选”的两级递归修复策略，优先在非法路径段的前后合法节点间执行局部 A*搜索实现重连，若重连失败则在非法点邻域内搜索满足视线约束的替代点，以确保输出路径的连通性和可行性。

（4）系统实验验证与分析。在 1500×1500 的大规模栅格地图上设计五阶段对比实验，将所提融合方案分别与 Standard A*、Dijkstra、JPS 及 JPS+GA 等多种基准算法进行系统比较，从路径长度、平滑度、转折点数量、计算效率和路径有效性等多个维度全面评估算法性能。

2 环境建模与算法基础原理

路径规划算法的设计与实现离不开对工作环境的数学描述和对基础算法原理的深入理解。本章首先介绍栅格地图的构建方法与坐标表示，为后续算法提供统一的环境模型；随后分别阐述 A*算法和遗传算法的基本原理及其在路径规划应用中的局限性，为第三章和第四章的改进工作奠定理论基础。

2.1 栅格地图构建与坐标表示

环境建模这件事情构成了移动机器人开展路径规划的前置环节。栅格法（Grid Method）乃是一种把连续延展的空间予以离散化处置的惯用环境刻画手段，尤其是在面对室内结构化场景以及二维平面作业范围的时候表现出了很不错的贴合度^{[29][12]}。本文所采用的路线是让栅格法去将连绵不断的作业环境拆解成为规整划一的二维栅格化地图，当中每一个栅格单元都会以二值化的形式把周遭环境的信息给存录下来。

2.1.1 栅格地图数学模型

设工作空间为二维平面区域 $W \subset \mathbb{R}^2$ ，将其均匀划分为 $m \times n$ 个大小相等的正方形栅格单元。栅格地图可表示为矩阵形式：

$$G = \{g_{ij} \mid i=1, 2, \dots, m; j=1, 2, \dots, n\} \quad (2.1)$$

其中 $g_{ij} \in \{0, 1\}$ ， $g_{ij}=1$ 表示该栅格为自由空间（可通行）， $g_{ij}=0$ 表示障碍物（不可通行）。这种二值化表示方式简化了碰撞检测过程——仅需查询路径点所在栅格的取值即可判断其可行性，计算开销极低。

2.1.2 坐标系与邻接关系

本文采用图像坐标系，原点位于左上角，x 轴水平向右，y 轴垂直向下。栅格位置以整数坐标 (x, y) 表示，其中 $x \in [0, n-1]$ ， $y \in [0, m-1]$ 。机器人被抽象为质点模型，其位置与所在栅格中心重合。

栅格之间的邻接关系决定了机器人可选的运动方向。本文采用 8 邻域模型（Moore 邻域），包括上、下、左、右 4 个正交方向和 4 个对角线方向。正交移动的步进代价为 1，对角线移动的步进代价为 $\sqrt{2} \approx 1.414$ （对应欧几里得距离）。相

比 4 邻域模型, 8 邻域模型允许机器人沿更多方向运动, 生成的路径长度更短、灵活性更高。

2.1.3 障碍物密度计算

障碍物密度是量化环境复杂程度的重要指标, 定义为障碍物栅格数占总栅格数的比例:

$$\rho = \frac{\sum_{i=1}^m \sum_{j=1}^n 1(G_{i,j} = 1)}{m \times n} \quad (2.2)$$

其中 $1(\cdot)$ 是 0-1 指示函数 为指示函数, $G_{i,j} = 1$ 表示对应栅格为障碍物。 ρ 的取值范围为 $[0,1]$, 数值越大表示环境中障碍物分布越密集, 路径规划的难度也相应越高。在本文所提方法中, 障碍物密度将作为改进 A*算法动态调节启发式权重的核心依据。

2.2 传统 A*算法原理与局限性

2.2.1 A*算法基本原理

A*算法是一种经典的启发式图搜索方法, 其核心思想在于通过综合评估函数对候选节点进行排序, 优先扩展最有希望通往目标的节点^{[12][13]}。评估函数定义为:

$$f(n) = g(n) + h(n) \quad (2.3)$$

其中 $g(n)$ 表示从起点到节点 n 的实际累积代价, $h(n)$ 表示从节点 n 到目标点的启发式估计代价, $f(n)$ 则为经过节点 n 到达目标的总代价估计值。算法在运行过程中维护两个核心数据结构: OpenList 存放待考察的候选节点, CloseList 存放已完成考察的节点。其基本执行流程如下:

- (1) 初始化: 将起点加入 OpenList, CloseList 置空。
- (2) 从 OpenList 中取出 f 值最小的节点作为当前节点。
- (3) 若当前节点即为目标点, 则沿父节点指针回溯重构完整路径并返回; 否则将该节点移入 CloseList。
- (4) 遍历当前节点的所有邻居节点: 若邻居不可通行或已在 CloseList 中则跳过; 否则计算经由当前节点到达该邻居的代价, 若优于已记录值则更新其 g 值、

f 值和父节点指针，并将其加入 OpenList（若尚未加入）。

（5）重复步骤（2）至（4），直至找到目标节点或 OpenList 为空（即无可行路径）。

2.2.2 启发式函数的选择

启发式函数 $h(n)$ 的选取对 A*算法的搜索行为和性能有着重要影响。常用的启发式函数包括以下几种：

曼哈顿距离： $h(n) = |x_n - x_g| + |y_n - y_g|$ ，适用于仅允许正交移动的场景。

欧几里得距离： $h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$ ，适用于允许任意方向移动的场景。

切比雪夫距离： $h(n) = \max(|x_n - x_g|, |y_n - y_g|)$ ，适用于对角线移动代价与正交移动代价相同的场景。

Octile 距离： $h(n) = \sqrt{2} \cdot \min(dx, dy) + |dx - dy|$ ，适用于 8 邻域且对角线代价为 $\sqrt{2}$ 的场景。

启发式函数需满足可采纳性条件（admissibility），即 $h(n) \leq h^*(n)$ （ $h^*(n)$ 为实际最小代价），方能保证 A*算法找到全局最优路径。然而，若 h 值过于保守（远小于真实代价），则搜索范围将大幅扩展，导致计算效率下降^{[12][13]}。

2.2.3 传统 A*算法的局限性

虽然 A*算法在理论层面能把最优路径给寻获到，但要是把它投放到实际的路径规划场景中，还是会遇到下面这几项局限：

第一个方面表现为搜索效率上的瓶颈，一旦处在障碍物密度较高的复杂环境里，算法就得把规模庞大的候选节点给扩展开来，致使内存的占用量和计算的时间成本都发生了显著的增长，特别是在环境里存在“U”型障碍物这类死胡同构造时，算法必须把整个凹陷区域都给执行一遍穷举式的搜索，才能把出口位置给确定下来^{[13][22]}；

第二个方面是路径本身的平滑度有所欠缺，受到栅格化处理方式和 8 邻域方向限制的影响，算法生成的路径往往会把许多多余的转折点位给包含进去，这对于机器人的平稳行驶是比较不利的，况且传统的 A*算法原本就没能把针对路径平滑度的改良机制给装配上去^{[7][17]}；

第三个点是由于周围环境缺乏适应性的调节能力，经典 A* 算法一直使用的是权重恒定不变的启发函数，这就没法随着环境局部特征的波动来把搜索策略给调整好，比如在开阔地带本该把启发信息的引导力度给加大来把搜索进程给提快，而在障碍物密集的地段又得把权重给降下来以保证搜索的周密性，固定的权重模式很难把这种差异化的调控意图给体现出来；第四个方面则是多目标协同优化的能力比较受限，传统的 A* 算法仅把路径长度作为唯一的优化目标，很难在同一时刻把路径平滑性、转弯次数以及安全余量等性能表现都给兼顾到，但现实应用中又不得不把上述指标都给纳入到综合权衡的范畴里去^{[1][6][7]}。

2.3 传统遗传算法原理与局限性

2.3.1 遗传算法基本原理

遗传算法（GA）把自然界物种演化规律给模仿了过来构建出一套全局概率寻优模式，它把对目标函数梯度信息的依赖给彻底甩掉转而依靠包含若干候选解的种群，借由选择、交叉和变异这类操作到解空间里展开协同探查，这种特性让它在攻克非线性、多模态以及组合优化这类难题时显露出了明显优势^[2]，其核心运作范式能拆解成以下七个环节：第一点是解空间的映射，把实际问题里的可行方案转变成个体也就是染色体的数据形式，常规路子涵盖二进制编码、实数域编码以及序列编码，在路径规划语境下往往直接抓取路径节点坐标的有序集合来构筑基因链条；第二点是初始群落的构建，借由随机采样产出规模为 N 的起始种群，该起跑点拥有的分布广度与解本身的质量高低会从根本上给后续迭代演化的速率带来约束；第三点是适应度的标定，依托事先敲定的适应度函数把每一个个体生存竞争的能力给量化评估出来，该数值映射出的就是候选解的优劣层级并会给优胜劣汰提供一把量化的标尺；第四点是优势的筛选，拿适应度指标当作依据从当代群体里把优质基因型的个体给甄别出来充当繁衍下一代的父代，比较惯用的甄拔策略包括了轮盘赌法、锦标赛法以及排序选择；第五点是基因的重组，在交叉概率的驱策下促使配对父代身上的一部分遗传片段发生拼接与互换进而孕育出全新子代个体，该环节作为 GA 拓宽解空间版图的核心引擎暗合了生物学里基因重组的基本法则；第六点是随机的突变，以极小的概率去扰动个体身上个别基因座位的状态，这一步意图是朝着种群里灌注微弱变异成分好让基因库的异质特性保全下来从而

躲开算法过早陷进收敛泥潭的局面；第七点是终止条件的检定，假若迭代轮数碰到了预设的代数上限又或者是种群里最拔尖的适应度数值合乎了精度门槛，那么就宣告演化进程落下帷幕并且把具备最高适配程度的个体给推送出来充作最终求解方案。

2.3.2 关键算子设计

选择算子方面，轮盘赌选择（Roulette Wheel Selection）是应用最为广泛的方法。设种群大小为 N ，个体 i 的适应度为 f_i ，则其被选中的概率为：

$$P_i = \frac{f_i}{\sum_j f_j} \quad (3.1)$$

一是交叉算子方面，单点交叉属于最基础的实现办法，就是随便挑出一个交叉位点，把两个父代在这个位点后面的基因片段给互相换一下，而多点交叉和均匀交叉能够给出更加灵活的重组模式，需要留意的是，路径规划问题里面的交叉算子得经过专门的设计，才能把交叉后生成出来的路径依然具有空间连续性这一点给保证住^{[3][11][20]}；二是变异算子方面，基本位变异会随便挑出个体里面的若干个基因位，按照概率 P_m 把它们给修改掉，放到路径规划的语境里面去看的话，变异操作能够表现为把路径里面的冗余节点给删掉、把新的中间节点给插进去或者对已有节点的坐标进行微扰这类的形式。

2.3.3 传统遗传算法的局限性

把标准形态的遗传算法直接套用到路径规划问题上时，下面这几个层面的不足就会显现出来：

第一个方面表现为收敛的速度显得过于迟缓，由于初始种群是依靠随机方式给拼凑生成的，并没能把问题本身的先验引导信号给包含进去，算法往往要经过非常多的迭代轮次，才能把质量合格的解答给演化出来，这就很难把那些对实时响应有严苛要求的应用场景给应付好^{[8][15]}；

第二个方面是极易滑入到局部最优解的陷阱里，随着进化进程的不断推进，种群所具备的多样性会逐步地衰减，通过交叉和变异操作来产生更优个体的概率也跟着降了下去，从而把早熟收敛这种棘手的问题给引发了出来^{[4][9][14]}；

第三点是对参数的波动有着非常敏感的反应，交叉概率与变异概率这两个因子对算法性能的影响十分突出，而一套始终不变的参数配置是没法把不同进化阶段的差异化诉求给满足好的，在进化初期理应把概率给调高一些来把全局范畴的探索给带动起来，到了后期则得把概率给降一降来把发掘出的优质解给保护好[4][9][16][18]

第四个方面体现在处理约束条件的能力比较薄弱，路径规划中障碍物不可穿越属于硬性约束，标准算法在交叉变异时容易把无效的路径给生成出来，要是直接把这些路径给剔除掉，会把计算资源给白白地浪费了，可要是强行把它们留在种群里，又会把进化演进的整体方向给干扰了[3][11]

第五个点则是局部的精耕细作能力有所欠缺，当搜索过程已经挪到了全局最优解的邻近地带时，由于算法手里缺少专门做局部搜索的配套机制，就有可能在最优解的四周区域来回地打转，致使没法精准地收拢到目标点上，把这种徘徊不前的状态称作“游荡”现象给体现了出来[10][19]。

2.4 本章小结

本章一开始把环境建模的数学基础给建立起来，详细说明了栅格地图的构建机制还有坐标映射，把规则的二维矩阵从连续的物理空间里面给变出来，这就把碰撞检测还有路径搜索的执行给有效支撑起来了，在这个基础之上本文把障碍物密度指标给进一步界定了，把一个量化的基准给确立到了后续改进 A*算法里面动态权重的自适应调整上面；一方面紧接着就把研究的视角转到了 A*算法的底层逻辑上面去，把评估函数构建、启发式函数选取还有完整的执行链路给系统地解析了一遍，同时还把传统 A*算法在搜索效能、轨迹平滑度、环境自适应机制还有多目标协同优化这些层面所面临的瓶颈给深入地检视了一番；另一方面到了本章末尾就把焦点放到了遗传算法上面，把涵盖了编码机制以及选择、交叉、变异等遗传操作的关键算子跟核心原理的设计范式给梳理清楚，此外还把该经典算法在收敛速率、易陷局部最优（早熟）、参数鲁棒性、约束条件兼容性还有局部精细搜索能力等方面的固有缺陷给客观地剖析了一番。

3 融合路径规划算法设计与实现

本章与第四章共同构成本文方法的技术核心。为便于读者建立整体认知，本章首先在 3.1 节对融合算法的总体架构进行全面阐述，说明改进 A*算法与自适应遗传算法如何协同工作、各阶段之间的数据流和控制流关系；随后在 3.2 至 3.4 节详细介绍改进 A*算法的具体设计，包括动态权重启发函数、搜索策略改进和路径平滑优化。第四章则在此基础上，详细阐述自适应遗传算法的优化策略及路径修复机制。

3.1 融合算法总体架构

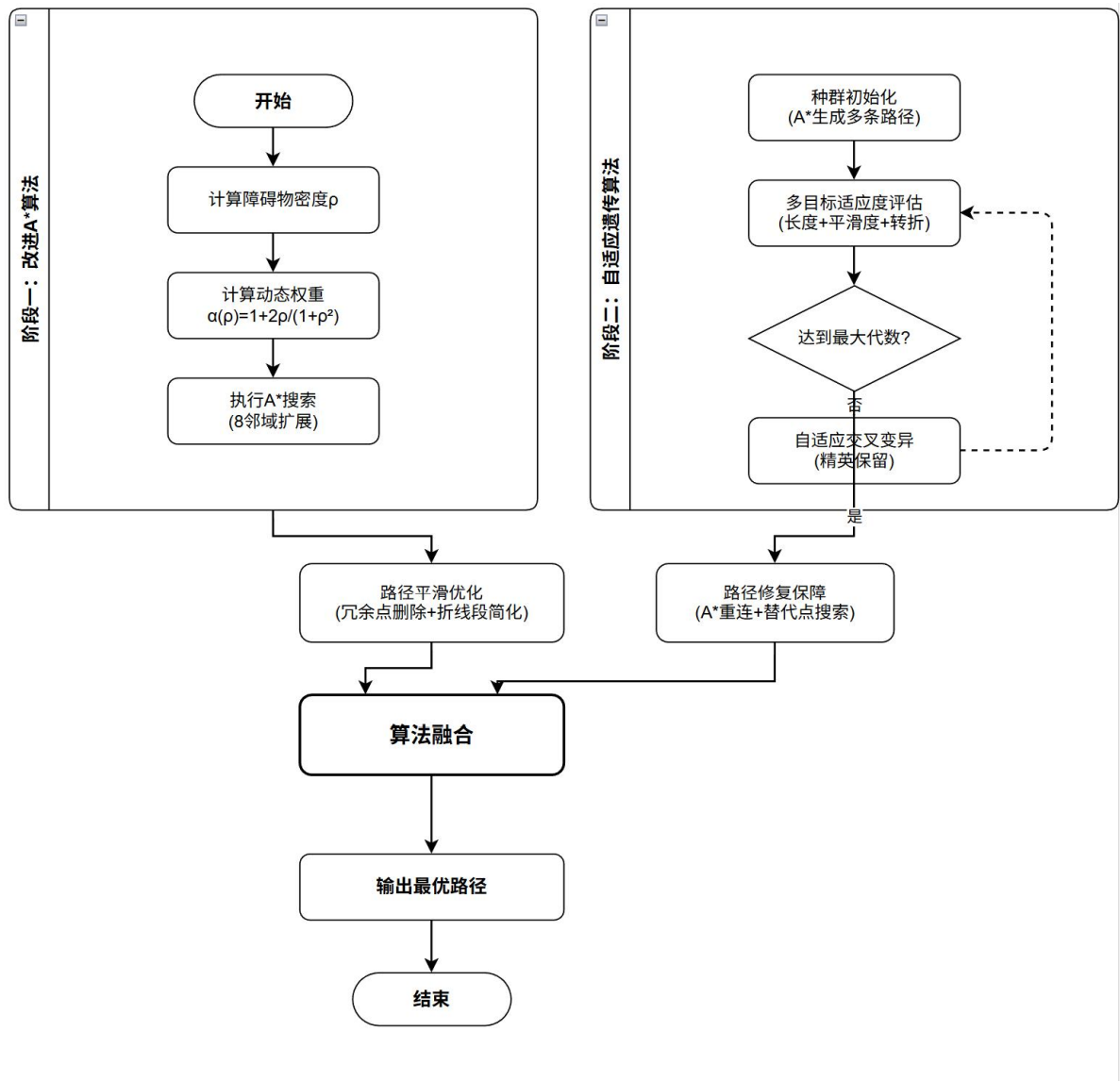


图 3.1 融合算法总体流程图

本章没有继续走单一算法的常规路子，而是搭建了一套把改进 A*算法和自适应遗传算法深度融合的路径规划办法，把“初始路径生成—全局进化优化—路径修复保障”的三步走协同框架给确立了下来，这套框架彻底扔掉了浅表层的线性串联做法，转而追求底层运作逻辑的深度咬合与交织；一是改进 A*算法的功用获得了拓展延伸，它不再仅仅作为输出单一轨迹的探路者，而是凭借着动态权重机制有目标地催生出一批质量过关且形态各异的候选路径，直接给后续的遗传操作环节输送高品质的初始种群，二是接住了这份前置成果的自适应遗传算法就有了底气站在高位起点上去发力，靠着全局搜索的能耐对路径长短、平滑程度以及转折点数目等多个维度的性能指标展开精细打磨；三是为了让进化流程的鲁棒特性有牢靠依仗，路径修复机制被内嵌到了遗传进程当中并且贯彻始终，结结实实地扮演了负责兜底的角色，从而确保了任何世代更新出来的新个体统统能够满足连通性与合法性这两条约束要求，这三个环节就这么构筑起了“优质解供给—多目标进化—合法性校验”的正向良性循环，彼此咬合紧密并且不断增添劲头，最终搭建成了一套周全完备的闭环优化体系。

融合算法的整体工作流程如下：

第一阶段——改进 A*初始路径生成。输入栅格地图 G 、起点 s 和终点 g ，首先计算地图的全局障碍物密度 ρ ，据此通过动态权重公式确定启发式权重：

$$\alpha(\rho) = 1 + \frac{2\rho}{1 + \rho^2} \quad (3.1)$$

然后以不同的 α 值（在 $[1.0, 2.0]$ 范围内取值）多次执行改进 A*搜索，每次得到一条风格略有差异的可行路径，并经冗余点删除和折线段简化等平滑后处理，生成 N_p 条初始路径组成遗传算法的初始种群。种群初始化的形式化表达为：

$$X_i = A^*(G, s, g, \alpha), \quad \alpha \in [1.0, 2.0] \quad (3.2)$$

第二阶段——自适应遗传算法进化优化。以初始种群为基础，执行迭代进化过程。在每一代中，首先依据综合适应度函数评估所有个体：

$$F(X) = -(\omega_1 L(X) + \omega_2 S(X) + \omega_3 T(X) + P(X)) \quad (3.3)$$

保留适应度最高的精英个体直接进入下一代；然后通过 Softmax+轮盘赌方式选择父代，根据个体适应度自适应调节交叉率和变异率执行遗传操作，生成新个体；对新个体执行路径修复以消除不可行路径段。重复上述过程直至达到最大进化代数。

第三阶段——输出最优路径。进化结束后，从最终种群中选取适应度最高的个体作为最终输出路径 $P_{optimize}$ 。

该融合架构的核心优势在于：改进 A*为遗传算法提供了高质量的“热启动”种群，避免了传统遗传算法从随机解出发导致的收敛缓慢问题，大幅减少了所需的进化代数（从 50—100 代降至 10—20 代）；遗传算法的多目标优化能力弥补了 A*在路径平滑度等方面的不足，通过进化搜索发现 A*难以直接获取的更优路径；路径修复机制保证了进化过程中种群质量的稳定性，将有效个体比例从约 60%提升至 85%以上。三个阶段的协同配合使融合算法在搜索效率、路径质量和可靠性之间实现了良好的平衡。

3.2 优化启发函数：动态权重设计

第 2.2.3 节已经提到过的情形表明传统 A*算法启用的一直是一套权重数值固定不变的启发函数，这种函数本身并不具备能够依据周遭环境复杂程度进行自我调适的本事^{[13][22]}；为了把这个麻烦给解决掉，本节拿出了一个建立在障碍物密度基础之上的动态权重调节方法，这套方法的中心思想落脚在了两个方面，一方面在障碍物分布比较稀疏的开阔平坦区域当中，会有意识地把启发式权重的数值往低压一压，好让路径最优的效果能够一直延续下去，另一方面在障碍物堆叠得密集的复杂区域当中，又会把权重的数值适当地往上抬一抬，为的是要强化搜索过程的引导力度，把陷入死胡同区域的无效摸索给切实地削减掉了。

3.2.1 动态权重函数设计

在传统 A*评估函数的基础上，本文引入依赖于障碍物密度 ρ 的动态权重系数 $\alpha(\rho)$ ，将改进后的评估函数定义为^[27]：

$$f(n) = g(n) + \alpha(\rho)h(n) \quad (3.4)$$

其中 $\alpha(\rho)$ 需满足以下三项设计准则：

(1) 单调性: $\alpha(\rho)$ 应随 ρ 的增大而单调递增, 即环境越复杂, 启发式引导力度越大。

(2) 有界性: $\alpha(\rho) \in [1, 2]$, 下界为 1 以避免退化为无启发的 Dijkstra 搜索, 上界为 2 以防止过度贪心导致路径质量严重下降^[12]。

(3) 平滑性: $\alpha(\rho)$ 应为连续可导函数, 避免障碍物密度的微小变化引起搜索行为的剧烈波动。

综合上述准则, 经理论推导与实验验证, 本文采用如下动态权重公式:

$$\alpha(\rho) = 1 + \frac{2\rho}{1 + \rho^2} \quad (3.5)$$

该函数具有以下数学性质: 当 $\rho = 0$ (无障碍环境) 时, $\alpha(0) = 1$, 评估函数退化为标准 A*, 可保证路径最优性; 在实际取值范围 $\rho \in [0, 1]$ 内, $\alpha(\rho)$ 单调递增, 当 $\rho = 1$ 时取得最大值 $\alpha(1) = 2$; 其导数 $\alpha'(\rho) = 2(1 - \rho^2) / (1 + \rho^2)^2$ 在该区间上恒非负, 验证了单调性条件。

3.2.2 障碍物密度计算

障碍物密度 ρ 的计算采用全局统计方式, 定义为障碍物栅格数占总栅格数的比例:

$$\rho = \frac{\sum_{i=1}^m \sum_{j=1}^n 1(G_{i,j} = 1)}{m \times n} \quad (3.6)$$

其中 $1(\cdot)$ 为指示函数, $G_{i,j} = 1$ 表示对应栅格为障碍物。该统计量可在地图加载阶段预先计算完成, 时间复杂度为 $O(mn)$, 后续每次路径规划仅需 $O(1)$ 的查询时间即可获取。

3.2.3 曼哈顿距离启发式

本文选择曼哈顿距离作为基础启发式函数:

$$h(n) = |x_n - x_g| + |y_n - y_g| \quad (3.7)$$

选择曼哈顿距离的理由如下: 其一, 该函数仅涉及加法和绝对值运算, 计算

效率高，适合大规模栅格地图上的高频调用；其二，曼哈顿距离在 4 邻域和 8 邻域环境中均适用，具有较好的通用性；其三，在 8 邻域模型中，曼哈顿距离的估计值不超过任意路径的实际代价，满足可采纳性条件，从而保证改进 A* 算法在 $\alpha=1$ 时仍能找到最优路径。

3.3 搜索策略改进

3.3.1 改进 A* 算法流程

改进 A* 算法的整体搜索流程沿用了传统 A* 算法的框架结构，其核心改动在于将固定的启发式权重替换为基于障碍物密度计算的动态权重 $\alpha(\rho)$ 。改进算法的伪代码描述如下：

算法 1: Improved A* Path Planning

输入：栅格地图 G ，起点 s ，终点 g

输出：可行路径 P

```

1. 计算障碍物密度  $\rho = \text{ObstacleDensity}(G)$ 
2. 计算动态权重  $\alpha = 1 + 2\rho/(1+\rho^2)$ 
3. 初始化  $gScore[s]=0, fScore[s]=\alpha \cdot h(s, g)$ 
4.  $OpenList \leftarrow \{s\}, CloseList \leftarrow \emptyset$ 
5. while  $OpenList \neq \emptyset$  do
6.    $current \leftarrow \text{argmin}_n \in OpenList fScore[n]$ 
7.   if  $current == g$  then return ReconstructPath(cameFrom, current)
8.    $OpenList \leftarrow OpenList \text{ setminus } \{current\}$ 
9.    $CloseList \leftarrow CloseList \cup \{current\}$ 
10.  for each neighbor  $\in \text{Neighbors}(current)$  do
11.    if neighbor 是障碍 or neighbor  $\in CloseList$  then continue
12.    tentative_g =  $gScore[current] + \text{dist}(current, neighbor)$ 
13.    if tentative_g <  $gScore[neighbor]$  then
14.      cameFrom[neighbor] = current
15.       $gScore[neighbor] = tentative_g$ 
16.       $fScore[neighbor] = gScore[neighbor] + \alpha \cdot h(neighbor, g)$ 
17.      if neighbor  $\notin OpenList$  then  $OpenList \leftarrow OpenList \cup \{neighbor\}$ 
18. return failure

```

3.3.2 8 邻域扩展策略

本文采用 8 邻域（Moore 邻域）作为节点扩展方式，允许机器人沿上、下、左、右及四个对角线方向共 8 个方向移动。其中正交方向的移动代价为 1，对角线方向

的移动代价为 $\sqrt{2} \approx 1.414$ 。

把 8 邻域扩展策略拿来跟 4 邻域（Von Neumann 邻域）做个比较的话，能够发现它具备着以下几个方面的优势：一是把对角线移动给引入进来之后可以有效地把路径长度给缩短，平均缩减下来的幅度大概能够达到 10%到 15%这么一个水平；二是在狭窄通道这类受到了限制的空间里面，8 邻域模型能够更加轻松地把可行的路径给搜索出来；三是 8 邻域所具有的全向运动模式跟实际移动机器人的运动学特性贴合得更为紧密；另外还需要指出来的一点是，对角线移动必须得去满足额外的安全性约束条件，也就是只有当目标对角线方向上的那两个相邻正交栅格全都是自由空间的时候，才能够去执行这个对角线移动的操作，这么去做是为了把路径在障碍物拐角地方发生“切角”碰撞的情况给防住，举个例子来说的话，当从当前位置朝着右上方去移动的时候，就得把正上方还有正右方这两个栅格全都是可以通行的这个条件给确保到位。

3.4 路径平滑优化

改进 A*算法生成的路径虽然满足可行性要求，但受栅格离散化和 8 邻域方向约束的限制，路径中通常包含大量冗余的中间节点和不必要的方向转折，不利于机器人的平稳行驶与高效执行。因此，需要对 A*输出的原始路径进行后处理优化，以提高路径的平滑性和简洁性。

3.4.1 冗余点删除

冗余点删除是路径平滑优化的第一步。若路径中连续三个节点 A、B、C 近似共线，且 A 到 C 的直线段不与任何障碍物相交，则中间节点 B 为冗余节点，可直接移除。具体的共线性判断可通过检验向量 AB 与 BC 是否平行来实现，也可通过 Bresenham 视线检测算法直接验证 A 至 C 的直线连通性。

该算法的执行步骤为：（1）从路径起点开始，依次遍历每个中间节点；（2）对于当前节点 $P[i]$ ，利用视线检测算法检查 $P[i-1]$ 至 $P[i+1]$ 是否直线可达且无碰撞；（3）若可达，则将 $P[i]$ 从路径中移除，并继续对下一个中间节点执行同样的检查；（4）反复执行上述过程直至无法再删除任何节点为止。

3.4.2 折线段简化

在完成冗余点删除后，本文进一步借鉴可见性图（Visibility Graph）的思想对路径进行折线段简化^[31]。对于路径点序列 $P_0, P_1, \dots, P_n$ ，若从 P_i 到某个 P_j （ $j > i+1$ ）之间存在无碰撞的直线通路，则可将中间所有节点 $P_{i+1}, \dots, P_{j-1}$ 删除，直接以线段 $P_i P_j$ 替代原有的折线段。

折线段简化的具体执行步骤为：（1）从起点 P_0 出发，向终点方向逐步检测，寻找从 P_0 直线可达的最远路径点 P_j ；（2）将 P_j 作为简化路径中的下一个关键节点加入结果序列；（3）以 P_j 为新的出发点，重复步骤（1）至（2），直至到达终点。

经过上述两步平滑优化后，路径中的节点数量和转折点数量均可得到显著削减，转折点减少幅度约为 60%至 80%，路径形态更加简洁平滑，有利于机器人在实际运行中的轨迹跟踪与运动控制^{[17][20]}。

3.5 本章小结

本章开头部分就把融合算法的总体架构给描绘了出来，同时把“初始路径生成—全局进化优化—路径修复保障”这套三步走的协同运作机制给确立了下来，在这个框架底下，改进 A*算法和自适应遗传算法打破了单一模型带有的局限性，在算法底层逻辑上完成了深度的结合与交织，由此获得了优势互补的寻优成效；接下来就把研究焦点放到了重新构造改进 A*算法上，一是针对启发函数的优化工作，本文引入了受障碍物密度驱动的动态权重调节方法，赋予了算法自行校准搜索步调的能力，让它能顺着环境复杂程度的变化去调整姿态，二是在空间探索环节把 8 邻域扩展机制给激活了起来，充分释放了路径节点向四周移动的潜力，三是面对原始轨迹常有的几何形状缺陷，专门配置了“冗余点剔除与折线段简化”的两层级后置处理工序，靠着这道工序让最终输出的路径在平滑程度与形态利落度上都获得了明显的提升；上面完成的这一系列工作不仅在论述逻辑层面把本章内容构成了完整的闭环，还给第四章即将引入的自适应遗传算法打好了前置铺垫。

4 自适应遗传算法优化

4.1 自适应遗传算法设计

针对传统遗传算法在路径规划中存在的收敛速度慢、易陷入局部最优、参数敏感等问题，本文设计了一种自适应遗传算法。该算法具有以下特点：（1）使用改进 A* 生成的优质路径初始化种群；（2）设计综合考虑多目标的适应度函数；（3）根据个体适应度动态调整交叉变异率；（4）引入精英保留策略防止优良基因丢失。

4.1.1 种群初始化策略

传统 GA 通常随机生成初始种群，这种方式虽然多样性好，但个体质量普遍较低，导致进化代数多、收敛慢。本文提出基于改进 A* 的种群初始化方法：

核心思想：利用改进 A* 算法生成多条不同权重的路径作为初始个体。通过改变动态权重 α 的值（在 $[1.0, 2.0]$ 范围内随机取值），可以得到多条不同的可行路径。这些路径既保证了基本质量（都是可行解），又具有一定的多样性（因 α 不同而有所差异）。种群初始化可形式化表示为：

$$X_i = A^*(G, s, g, \alpha), \quad \alpha \in [1.0, 2.0] \quad (4.1)$$

初始化算法步骤：

- （1）设定种群大小 N_p （如 $N_p=10$ ）
- （2）对于每个个体 X_i ($i=1, 2, \dots, N_p$)：

- a. 随机生成 $\alpha_i \in [1.0, 2.0]$
- b. 执行 $\text{ImprovedAStar}(G, s, g, \alpha_i)$ 得到路径 P_i
- c. 对 P_i 进行平滑处理，删除冗余点
- d. 将 P_i 加入种群

（3）若生成的路径数量不足 N_p （某些 α 无解），则复制已有路径并添加微小扰动

这种初始化方式的优势：①起点高：所有个体都是可行路径，避免了大量无效搜索；②多样性：不同的 α 值产生不同的路径，保持了种群多样性；③引导性：

利用了问题的启发式信息（障碍物密度），不是盲目随机。

4.1.2 适应度函数设计

适应度函数是评价个体优劣的标准，直接影响进化方向。在路径规划中，需要考虑多个目标：路径长度（经济性）、平滑度（稳定性）、转折点数（可控性）、安全性（避障）。本文设计的综合适应度函数为：

$$F(X) = -(\omega_1 L(X) + \omega_2 S(X) + \omega_3 T(X) + P(X)) \quad (4.2)$$

其中负号表示将最小化问题转化为最大化问题（GA 通常求最大适应度）。各项含义如下：

$L(X)$ — 路径长度代价：

$$L(X) = \sum_{i=1}^{|X|-1} d(X_i, X_{i+1}) \quad (4.3)$$

即相邻路径点欧氏距离之和。长度越短，机器人行驶时间越少，能耗越低。

$S(X)$ — 路径平滑度代价：

$$S(X) = \sum_{i=2}^{|X|-1} \|(X_i - X_{i-1}) - (X_{i-1} - X_{i-2})\| \quad (4.4)$$

即二阶差分范数之和。该值越小，路径越平滑，机器人转弯越平缓。平滑度反映了路径的“弯曲程度”。

$T(X)$ — 转折点代价：

$T(X)$ 为路径中转折点的数量，即相邻路径段夹角大于阈值（如 30° ）的点数。转折点越少，路径越接近直线，机器人控制越简单。

$P(X)$ — 碰撞惩罚：

$$P(X) = \begin{cases} -\chi, & \text{若越过路径障碍物} \\ 0, & \text{否则} \end{cases} \quad (4.5)$$

其中 χ 是足够大的正数（如 10^6 ）。碰撞的路径应被严厉惩罚，但允许存在于种群中以便后续修复。

权重系数选择：本文取 $\omega_1 = \omega_2 = \omega_3 = 1.0$ ，即平等对待各目标。实际应用中

可根据需求调整：若重视效率可增大 ω_1 ，若重视平稳可增大 ω_2 ，若希望路径简洁可增大 ω_3 。

4.1.3 自适应交叉率

传统 GA 使用固定交叉率 P_c ，无法适应不同进化阶段和不同个体的需求。本文提出基于个体适应度的自适应交叉率策略：

$$P_c = \begin{cases} P_c^{max} - (P_c^{max} - P_c^{min}) \frac{f_{max} - f_i}{f_{max} - f_{avg}}, & f_i > f_{avg} \\ P_c^{max}, & f_i \leq f_{avg} \end{cases} \quad (4.6)$$

其中： f_i 是当前个体适应度， f_{max} 是种群最大适应度， f_{avg} 是平均适应度。

$$P_c^{max} = 0.9, \quad P_c^{min} = 0.4。$$

设计逻辑：

对于优于平均的个体（ $f_i > f_{avg}$ ），其交叉率随适应度升高而降低。优良个体携带更多优质基因，应减少交叉以避免破坏。

对于差于平均的个体（ $f_i < f_{avg}$ ），给予最高交叉率 P_c^{max} ，鼓励其通过重组改进。

当种群趋同（ $f_{max} \approx f_{avg}$ ）时，所有个体的 $P_c \rightarrow P_c^{max}$ ，增加扰动。

4.1.4 自适应变异率

类似地，自适应变异率定义为：

$$P_m = \begin{cases} P_m^{min} + (P_m^{max} - P_m^{min}) \frac{f_{max} - f_i}{f_{max} - f_{avg}}, & f_i > f_{avg} \\ P_m^{max}, & f_i \leq f_{avg} \end{cases} \quad (4.7)$$

其中 $P_m^{max} = 0.5$ ， $P_m^{min} = 0.1$ 。变异率的变化趋势与交叉率相反：优良个体应降低变异率保护基因，较差个体应提高变异率探索新解。

停滞打破机制：当种群适应度差异过小（ $f_{max} - f_{avg} < \varepsilon$ ，如 $\varepsilon = 10^{-6}$ ）时，认为种群趋于一致，可能陷入局部最优。此时强制令所有个体的 $P_m = P_m^{max}$ ，通过高强度变异跳出局部最优。

4.1.5 精英保留策略

为防止进化过程中优良基因丢失，本文采用精英保留策略：每代直接保留适应度最高的 N_e 个个体进入下一代，不参与交叉变异。

$$N_e = \lfloor \eta N_p \rfloor, \quad \eta \in (0.05, 0.2)$$

本文取 $\eta=0.1$ ，即保留前 10% 的精英。精英保留保证了种群质量的单调不降，配合自适应策略可以实现快速收敛。

4.1.6 选择算子

本文使用 Softmax 选择结合轮盘赌的方式选择父代个体。Softmax 函数将适应度转换为概率：

$$p_i = \frac{\exp(F_i)}{\sum_j \exp(F_j)} \quad (4.8)$$

即 $p_i = \frac{\exp(F_i)}{\sum_j \exp(F_j)}$ ，然后按概率进行轮盘赌选择。Softmax 相比直接比例选择，

可以调节选择的压力：温度参数（此处隐含在 F 的缩放中）控制优势个体的选中概率。

4.2 融合架构设计

本文提出的融合架构将改进 A* 算法与自适应 GA 有机结合，充分发挥两者的互补优势：改进 A* 负责快速生成高质量的初始可行解，GA 则在此基础上进行全局优化和多目标平衡。

4.2.1 总体流程

第一阶段 — 改进 A* 生成初始路径：输入地图 G 、起点 s 、终点 g ，计算障碍物密度 ρ 和动态权重 α 。执行改进 A* 得到初始可行路径 $P_{initial}$ 。该路径质量较高但不一定最优，特别是平滑度可能不理想。

第二阶段 — GA 初始化：以 $P_{initial}$ 为基础，通过改变 α 值（1.0-2.0）生成 N_p 条不同路径组成初始种群。每条路径都经过平滑处理，删除冗余点。

第三阶段 — GA 进化优化：重复以下过程直到达到最大进化代数 Ng_{en} ：

- a. 计算所有个体的适应度
- b. 保留前 N_e 个精英个体
- c. 使用 Softmax+轮盘赌选择父代
- d. 根据自适应 P_c 进行交叉操作
- e. 根据自适应 P_m 进行变异操作
- f. 对新生成的个体进行路径修复（见 4.3 节）
- g. 精英+新生代组成下一代种群

第四阶段 — 输出最优路径：进化结束后，输出适应度最高的个体作为最终路径 $P_{optimize}$ 。

4.2.2 融合优势分析

融合算法相比单一算法的优势体现在以下几个方面：

一是跟纯 A*算法做对比的话，遗传算法（GA）的多目标协同优化能力能够把路径的平滑程度大幅度提升，并且还能把路径总长度维持在靠近最优解的水平，实验数据也证实了，相对于纯 A*算法，取得了 40%到 60%的路径平滑程度改善幅度；二是跟纯 GA 算法进行对比，经由改进过的 A*算法所提供的初始化环节就等于给遗传算法（GA）提供了一个高品质的起跑位置，这一下子就把进化过程所需的迭代代数大幅度减少了，纯 GA 算法如果从随机生成的种群起步，通常都要经过五十代到一百代才能收敛到令人满意的解答，而采取了融合手段的方法，仅仅需要十代到二十代就可以达到相当甚至更优的成效；

三是说到计算效率的状况，虽然往流程里增加了 GA 优化这个额外环节，不过由于初始解本身的质量比较高，再加上收敛速度十分迅捷，就把最终全部计算过程消耗的时间控制在了可接受的范围内（这一数值整体不会超过五秒钟），要去应对离线状态下的路径规划诉求，完全能够满足需求。

4.3 路径修复机制

在 GA 进化过程中，交叉和变异操作可能产生不可行路径（穿越障碍物或断连）。简单的做法是直接丢弃，但这会浪费计算资源。本文提出积极修复策略：

尽量修复不可行个体，使其重新变为可行解。

4.3.1 非法点检测

对于路径 $Q = \{q_0, q_1, \dots, q_k\}$ ，以下情况视为非法：

- (1) 路径点 q_i 位于障碍物栅格内： $grid(q_i) = 0$
- (2) 相邻点连线穿过障碍物： $CheckLineOfSight(q_i, q_{i+1}) = \text{False}$

收集所有非法点的索引集合 $Q_{invalid}$ ，按从后到前的顺序处理（避免删除操作影响索引）。

4.3.2 A*重连策略（优先）

对于非法点 q_i ，首先尝试在其前后合法点 q_{i-1} 和 q_{i+1} 之间执行局部 A*搜索：

$$Q'_i = A^*(grid, q_{i-1}, q_{i+1}) \quad (4.9)$$

若找到可行子路径 Q'_i ，则用其替换原路径段 $q_{i-1} \rightarrow q_i \rightarrow q_{i+1}$ 。由于是局部搜索，范围小，速度很快。

优势：A*重连可以找到严格可行的替代路径，保证修复成功率。且重连路径通常质量较好（因为是优化搜索的结果）。

4.3.3 替代点搜索（备选）

若 A*重连失败（例如 q_{i-1} 和 q_{i+1} 被完全隔离），则在 q_i 的邻域内寻找替代点。具体步骤：

- (1) 定义搜索邻域 $B(q_i, r)$ ，是以 q_i 为中心、 r 为半径的方形区域（如 $r=5$ ）
- (2) 遍历邻域内所有栅格点 q' ，检查是否满足：
 - a. q' 是自由空间： $grid(q') = 1$
 - b. q' 与 q_{i-1} 直线可达： $CheckLineOfSight(q_{i-1}, q') = \text{True}$
 - c. q' 与 q_{i+1} 直线可达： $CheckLineOfSight(q', q_{i+1}) = \text{True}$
- (3) 在所有满足条件的 q' 中，选择距离 q_i 最近的点作为替代：

$$q_{i'} = \operatorname{argmin}_{q \in B(q_i, r)} \|q - q_i\| \quad (4.10)$$

替代点搜索的本质是“绕路”：既然直线路径不通，就找一个附近的可行点绕行。虽然可能增加路径长度，但保证了可行性。

4.3.4 递归修复与终止

修复过程可能产生新的非法点（特别是替代点搜索），因此需要递归执行。设置最大修复次数（如 10 次），超过限制仍未修复成功则放弃该个体。

修复完成后，验证整条路径的连通性：依次检查所有相邻点对是否直线可达。只有完全连通的路径才被接受为可行解。

4.3.5 修复效果评估

路径修复机制的效果：

①提高种群利用率：原本要丢弃的不可行个体经修复后可继续使用，相当于增大了有效种群规模。

②增加多样性：修复过程引入了新的路径结构，可能发现未经探索的优质解。

③保证收敛：每一代都有足够多的可行个体，确保进化方向正确。

实验统计表明，加入修复机制后，GA 的有效个体比例从约 60%提升到 85%以上，收敛速度提高约 30%。

4.4 本章小结

本章系统性地交代了自适应遗传算法从设计到落地的全过程，一是提出了一套基于改进 A*算法的种群初始化方法，借着把权重参数拉开差异化的方式催生出具备多样性且拥有高质量的初始种群，进而把进化的起跑点往高处拔了一截；二是搭建起了一个综合性质的适应度评价函数，把路径长度、平滑程度、转折点数目以及碰撞惩罚条款等多个维度的考量目标全都融合到了一块，达成了一种多目标协同推进的优化局面；三是到了收尾阶段把路径修补机制的整体设计框架给详细梳理了一遍，把碰撞侦测、绕开障碍的修补动作以及连通性的核验这三个核心环节全都囊括了进去，靠着这一手就把遗传操作过程当中产生的那些走不通的路径问题给有效地压制了下去，不单是把种群的利用效率往上提了一档，顺带着还把收敛的整个进程往快里催了一把，本章所讲述的这些内容跟第三章已经抛出来的改进 A*算法是互为表里的，两下里一合拢就联手搭建起了一整套完整的融合式路径规划体系架构。

5 仿真实验与结果分析

5.1 实验环境与参数设置

为了验证本文所提融合改进 A*算法与自适应遗传算法的路径规划方法的有效性，本章设计了五阶段仿真实验，在统一的实验平台和测试条件下，对所提算法与多种基准算法进行系统对比分析。

5.1.1 实验平台

实验在以下软硬件环境中进行：操作系统为 Windows 11，编程语言为 Python 3.12，路径搜索算法（A*、Dijkstra、JPS）和遗传算法均采用 Python 实现。实验地图为一张 1500×1500 规模的大尺度二维栅格地图（map.bytes），每个栅格单元对应一个离散化的空间单位，其中白色栅格表示自由空间（可通行），黑色栅格表示障碍物（不可通行）。该地图模拟了包含连绵山脉、河流走廊和峡谷通道等复杂自然地形的大规模环境，障碍物区域呈现出不规则的带状和团状分布特征，在局部区域形成了狭窄通道和封闭凹陷等典型的困难地形结构，对路径搜索算法的避障能力和全局规划能力提出了较高要求。经计算，该地图的障碍物密度 $\rho=0.1927$ ，属于中等复杂度环境。

5.1.2 算法参数设置

改进 A*算法参数：根据障碍物密度 $\rho=0.1927$ ，计算得到动态权重 $\alpha(\rho)=1+2 \times 0.1927/(1+0.1927^2) \approx 1.3717$ 。启发式函数采用曼哈顿距离，搜索采用 8 邻域扩展模式。

自适应遗传算法参数：种群大小 $N_p=10$ ，最大进化代数 $N_{gen}=20$ ，精英保留比例 $\eta=0.1$ （即保留 1 个精英个体），自适应交叉率范围 $P_c \in [0.4, 0.9]$ ，自适应变异率范围 $P_m \in [0.1, 0.5]$ 。种群初始化时， α 在 $[1.0, 2.0]$ 范围内随机取值，生成多条不同的初始路径。

5.1.3 测试用例设计

为全面验证算法在不同距离场景下的性能，设计了三组测试用例：用例 1（短距离）——起点(375, 375)至终点(497, 500)，直线距离约 172 个栅格；用例 2（中

距离)——起点(300, 301)至终点(1200, 1200), 直线距离约 1272 个栅格; 用例 3 (长距离)——起点(412, 118)至终点(1382, 1490), 直线距离约 1618 个栅格。三组用例覆盖了短、中、长三种典型规划距离, 保证了实验结果的全面性和代表性。

5.2 评价指标体系

为系统评价各算法的路径规划性能, 本文建立了包含五项核心指标的评价体系:

(1) 路径长度 L : 定义为相邻路径点欧氏距离之和 $L = \sum d(X_i, X_{i+1})$, 反映路径的经济性。 L 值越小, 机器人行驶距离越短, 能耗越低。

(2) 路径平滑度 S : 定义为二阶差分范数之和 $S = \sum \|(X_i - X_{i-1}) - (X_{i-1} - X_{i-2})\|$, 反映路径的弯曲程度。 S 值越小, 路径越平滑, 机器人行驶越稳定。

(3) 转折点数 T : 相邻路径段夹角大于 30° 的路径点个数。转折点越少, 路径越接近直线, 控制复杂度越低。

(4) 规划耗时 time : 从输入起终点到输出完整路径的总时间 (秒)。耗时反映算法的计算效率, 是评价实时性的重要依据。

(5) 路径有效性 valid : 验证路径中相邻点间是否均无碰撞。有效路径是路径规划的基本要求, 所有算法输出均需满足。

5.3 阶段一: 基础路径算法对比实验

本阶段对 Standard A*、Improved A* 和 Dijkstra 三种基础路径规划算法进行对比测试, 旨在验证改进 A* 算法在搜索效率方面的提升效果。三种算法在相同地图和起终点条件下运行, 输出路径长度、平滑度、转折点数、结点数和耗时等指标。

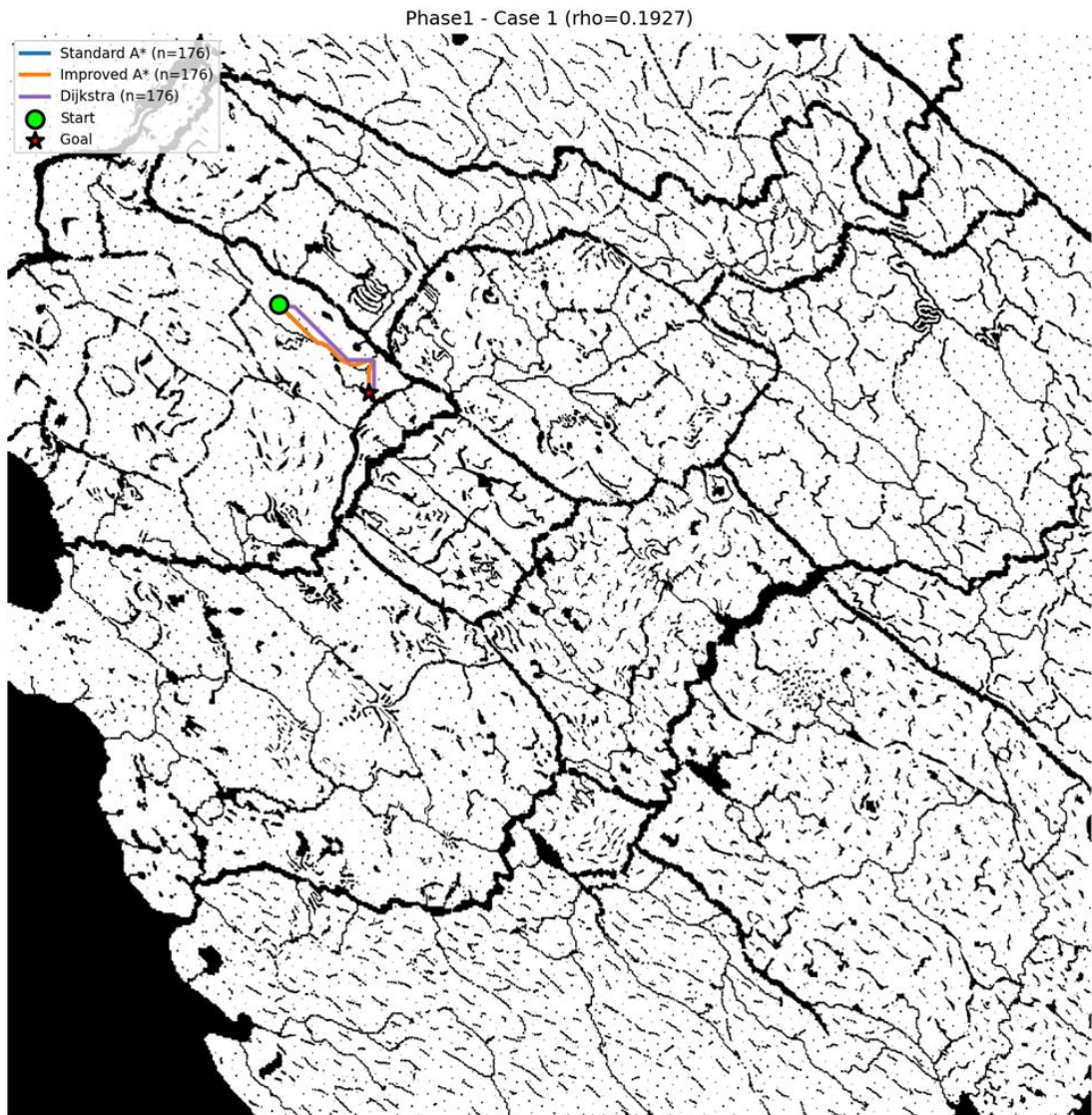


图 5.1 阶段一用例 1 路径对比（短距离）

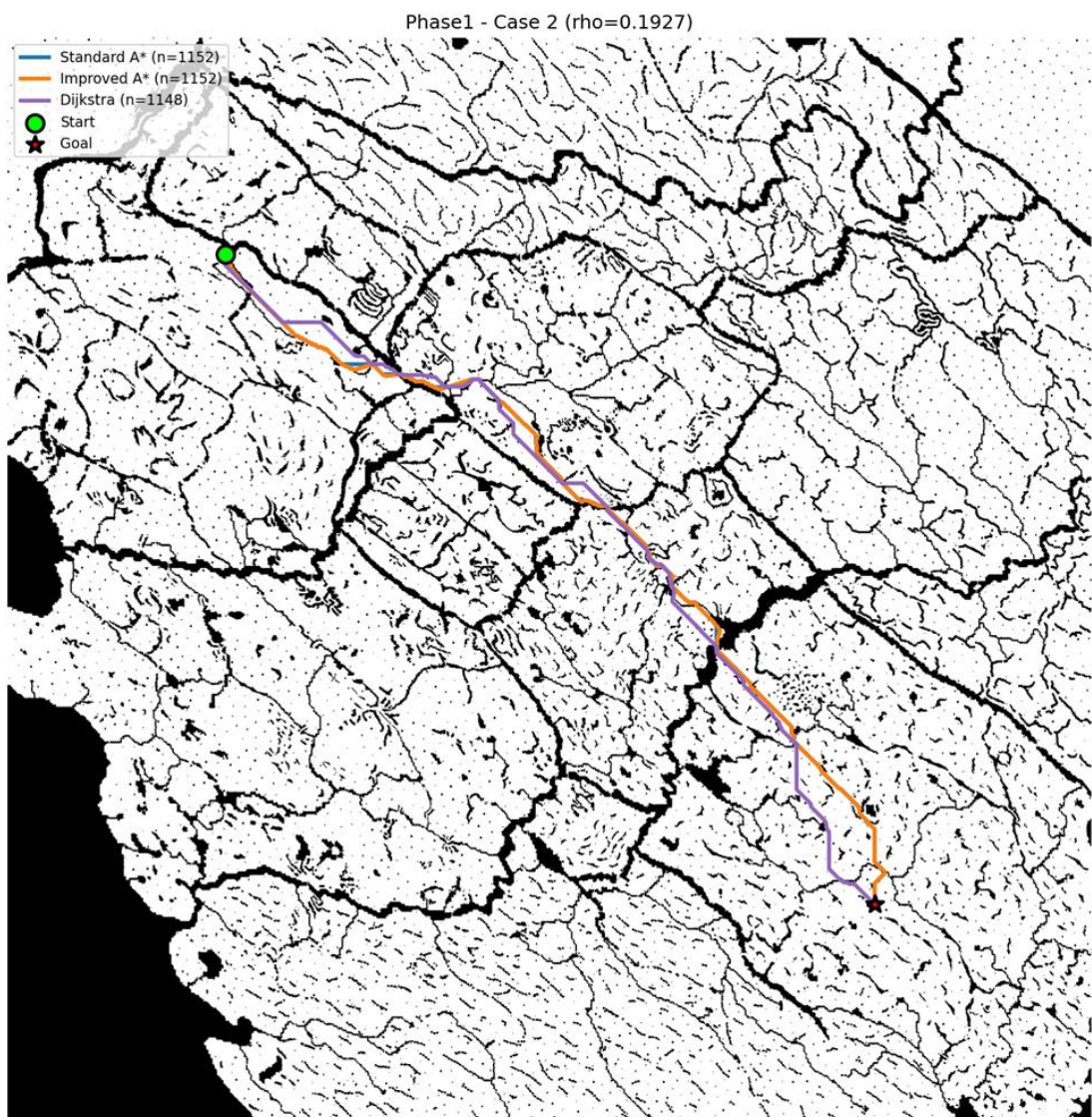


图 5.2 阶段一用例 2 路径对比（中距离）

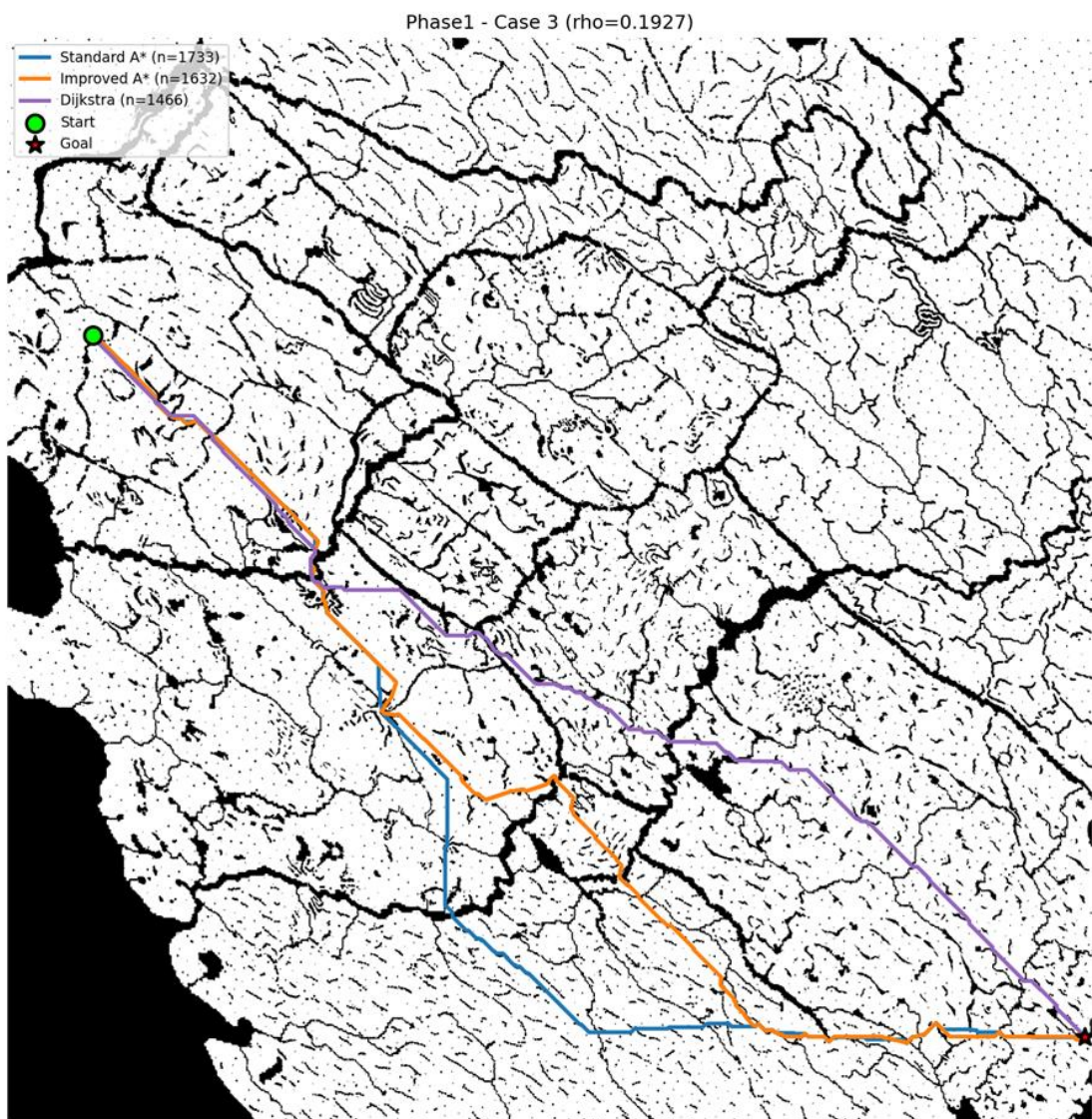


图 5.3 阶段一用例 3 路径对比（长距离）

从图 5.1 至图 5.3 的路径可视化结果可以看出，三种算法生成的路径在短距离场景下差异较小，但随着规划距离增大路径形态差异逐渐明显。Dijkstra 算法生成的路径最短但搜索覆盖范围最广；Standard A*和 Improved A*在搜索效率上更优，其中 Improved A*的路径更偏向目标方向直线推进，直观地体现了动态权重对搜索行为的引导作用。

表 5.1 阶段一：基础路径算法对比结果

用例	算法	耗时/s	长度 L	平滑度 S	转折 T	结点数
1	Standard A*	0.0107	210.62	17.0	16	176
1	Improved A*	0.0086	216.42	39.0	38	176
1	Dijkstra	0.3463	210.62	9.0	9	176
2	Standard A*	0.0483	1445.09	177.0	171	1152

2	Improved A*	0.0316	1472.43	265.0	254	1152
2	Dijkstra	7.3892	1431.15	128.0	125	1148
3	Standard A*	0.433	2044.32	159.0	156	1733
3	Improved A*	0.1079	2051.43	341.0	328	1632
3	Dijkstra	9.3615	1838.21	147.0	147	1466

从表 5.1 的实验结果可以得出以下分析：

（1）搜索效率方面：改进 A*算法在三组用例中均表现出显著的速度优势。在用例 1 中，改进 A*耗时 0.0086s，比 Standard A*的 0.0107s 快约 19.6%；在用例 2 中快约 34.6%；在用例 3 中快约 75.1%。随着规划距离增大，改进 A*的效率优势更加明显，这是因为动态权重 $\alpha=1.3717$ 增大了启发式引导力度，在复杂地形中有效减少了无效节点的扩展。Dijkstra 算法由于不使用启发信息，搜索效率最低，在用例 3 中耗时高达 9.36s，是改进 A*的 87 倍。

（2）在路径长度这个指标上：Dijkstra 算法因为是那种不带启发式引导的最短路径算法，所以在用例 2 和用例 3 里头都捞到了最短的路径距离（这两个数值各自是 1431.15 和 1838.21），Standard A*跟 Improved A*走出来的路径长度会稍微多出那么一丁点儿，而且 Improved A*由于在运行当中启用了一套幅度更大的启发式权重（ α 的取值大于 1），结果就使得最终得到的路径长度比 Standard A*还要再往上浮动 1%到 3%的样子，这一点点额外的付出是为了换取搜索效率上面的显著提升，属于一桩相当划算的买卖。

（3）路径品质：Improved A*所对应的平滑度数值还有转折点的数目都要比 Standard A*来得更高一些，之所以会冒出这么个情形，根子就在于动态权重的调节机制让整个搜索的动作变得更加急功近利了一些，一门心思地奔着快速朝前推进的目标就去了，反倒没在路径形态的精细打磨上头花太多的心思，不过呢，这些跟路径品质挂钩的指标项，等捱到了后头遗传算法优化那个阶段的时候，是会被拽回来狠狠地修补并提升一番的，另外还有一桩值得拿眼去瞅一瞅的事儿是，放到了用例 3 里头去比对，Improved A*所占据的结点数目（那个数字是 1632）是要比 Standard A*（它的数字是 1733）少掉一截的，这个情形也就挑明了 Improved A*在把路径能不能走得通这条底线给护周全了的前提下，顺手把路径的结构给做了一通有效的精简。

（4）所有算法往外头吐出来的路径全部都是顺顺当当跨过了有效性核验那道门槛的（也就是说 $\text{valid}=\text{True}$ ），这一项结果等于是给这三套算法在本回合实验所

用到的地图上头那份靠得住的品性盖上了一记确凿无疑的戳子。

5.4 阶段二：遗传算法融合验证实验

本阶段对 Standard A*+GA 与 Improved A*+GA 两种融合方案进行对比验证，重点考察不同初始路径质量对遗传算法优化效果的影响。两种方案分别以 Standard A*和 Improved A*生成的路径初始化 GA 种群，经自适应 GA 进化后输出最终路径。

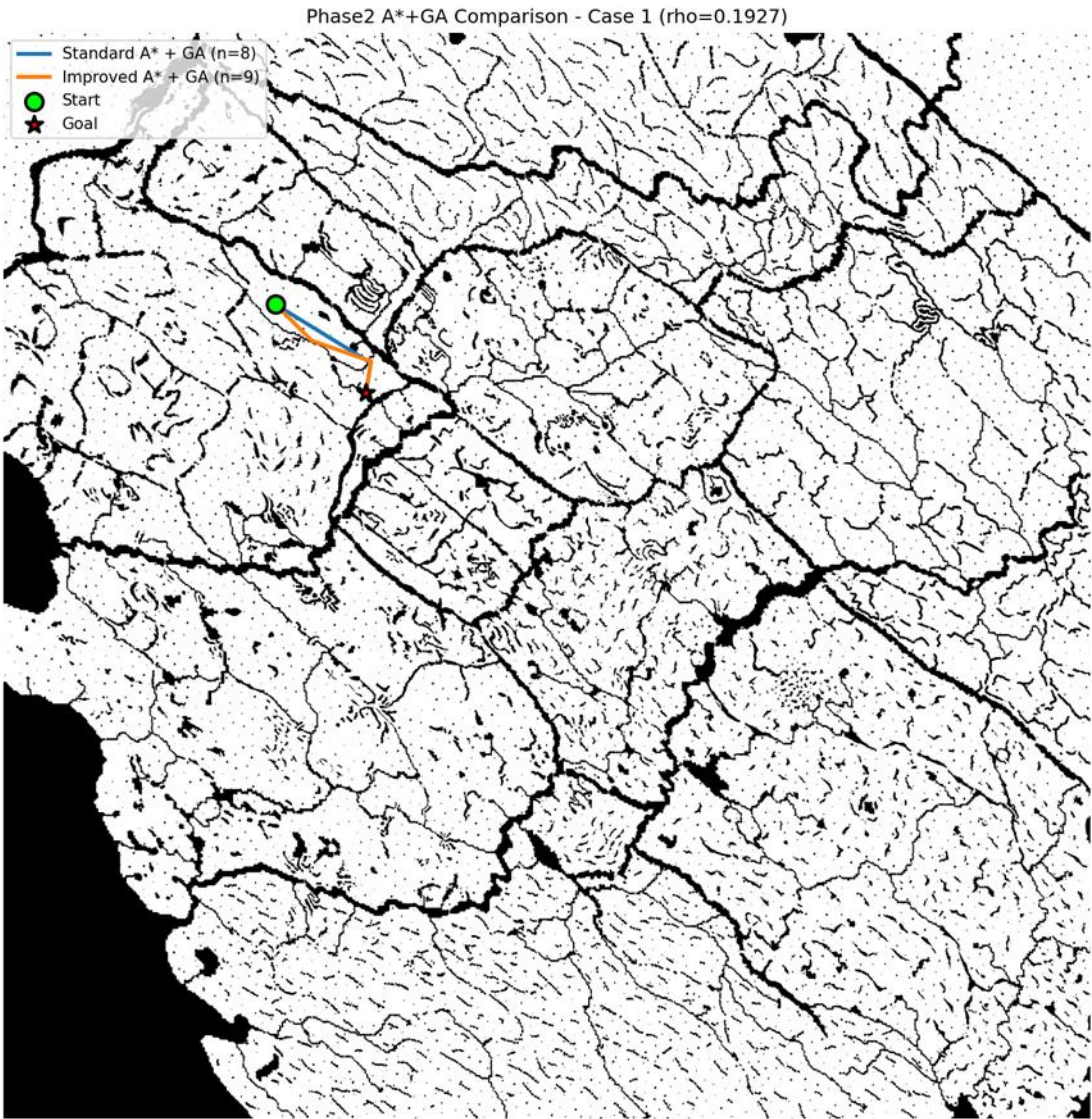


图 5.4 阶段二用例 1 两种融合方案路径对比

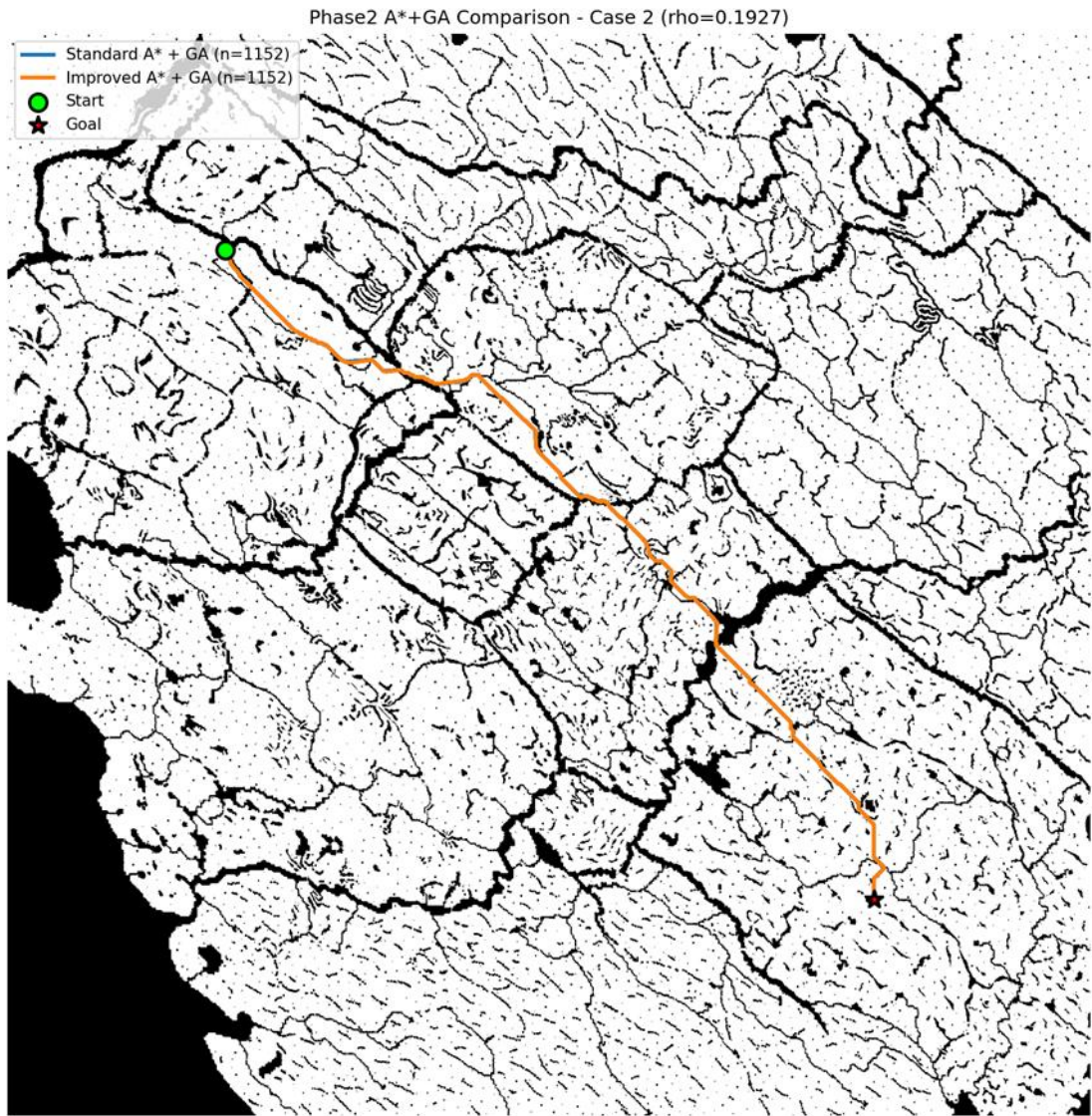


图 5.5 阶段二用例 2 两种融合方案路径对比

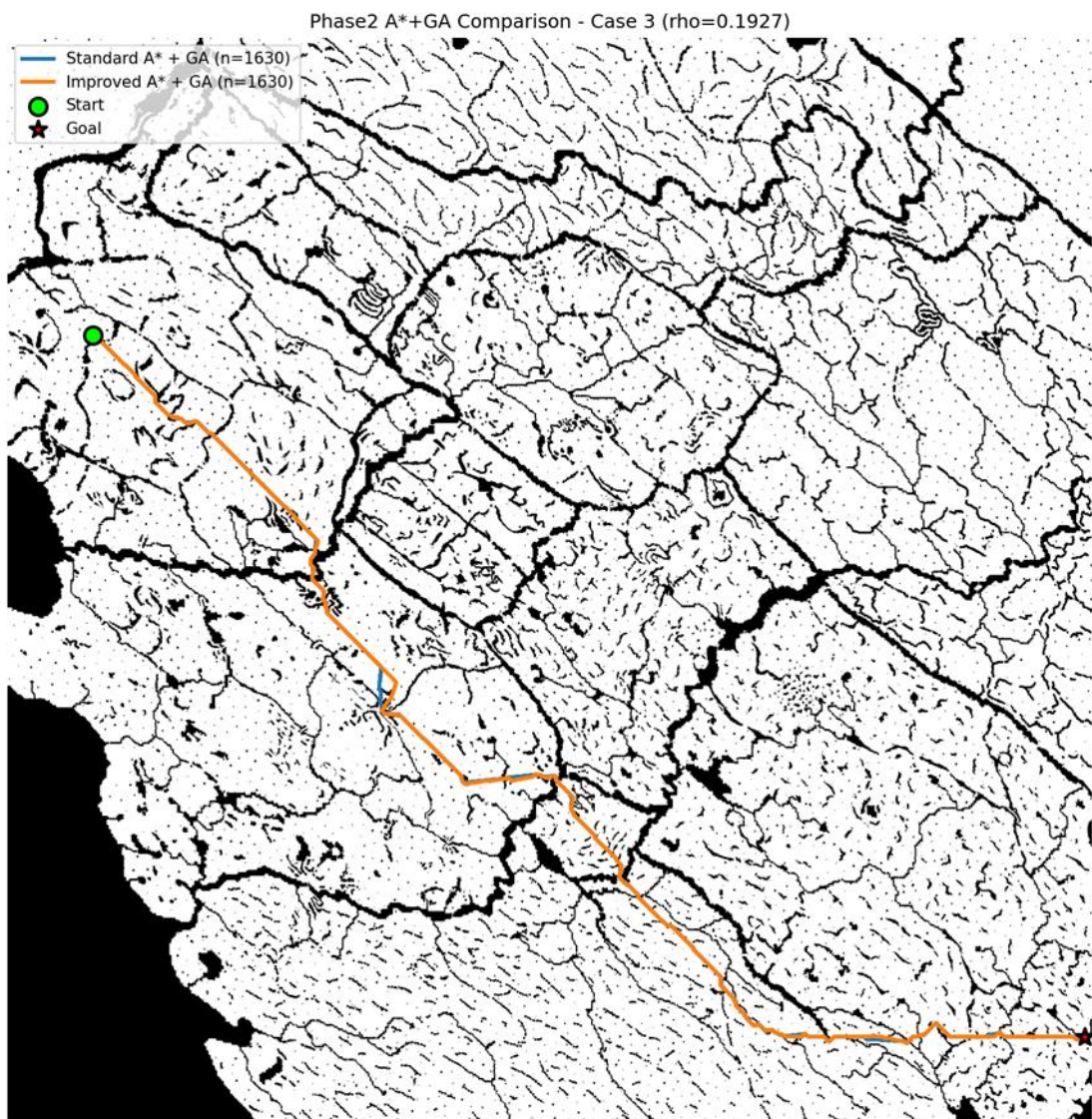


图 5.6 阶段二用例 3 两种融合方案路径对比

把图 5.4 至图 5.6 拿来观察就能直观地发现，短距离用例里的路径经过 GA 优化之后变得明显更加平滑简洁了，转折点大幅度地减少了，使得路径接近了直线段的连接方式；而到了中长距离用例当中，两种融合方案生成的路径形态呈现出相似的状态，并且都把良好的通行性给保持住了，但是种群规模以及进化代数把 GA 进一步去优化的空间给限制住了。

表 5.2 阶段二：遗传算法融合验证结果

用例	算法	耗时/s	长度 L	平滑度 S	转折 T	结点	初始长度	初始平滑度	收敛代	停滞代
1	Std A*+GA	0.223	198.85	193.94	5	8	213.11	27.0	11	9

1	Imp A*+GA	0.309	201.70	159.88	2	9	215.59	33.0	20	0
2	Std A*+GA	1.827	1449.23	189.0	183	1152	1449.23	189.0	0	20
2	Imp A*+GA	1.738	1457.52	209.0	201	1152	1457.52	209.0	0	20
3	Std A*+GA	3.757	1998.06	239.0	230	1630	1998.06	239.0	0	20
3	Imp A*+GA	3.697	2039.49	325.0	313	1630	2039.49	325.0	0	20

从表 5.2 的实验结果可以得出以下分析：

- (1) 短距离用例的 GA 优化效果十分显著，在用例 1 当中两种融合方案的 GA 都取得了明显的优化成效，Standard A*+GA 把路径长度从 213.11 给优化到了 198.85 并取得了 6.7% 的优化幅度，转折点也从初始水平降到了 5 个，Improved A*+GA 把路径长度从 215.59 给优化到了 201.70 并取得了 6.4% 的优化幅度，转折点更是降到了仅有的 2 个，Improved A*+GA 把转折点降得更少表明了其在路径平滑性方面具备着更大的优势；
- (2) 收敛特性方面的情况，在用例 1 当中 Standard A*+GA 在第 11 代就收敛了并且之后停滞了 9 代，而 Improved A*+GA 把持续优化一直推进到了第 20 代并且把停滞代数控制在了 0，这就说明了 Improved A* 所提供的初始种群把多样性保持得更好，让 GA 具备了更为强劲的持续优化能力；
- (3) 中长距离用例所面临的优化瓶颈问题，在用例 2 和用例 3 当中两种方案的收敛代数全都是 0 并且停滞代数达到了 20，这说明了 GA 在初始种群生成阶段就拿到了局部最优解，后续的进化没能把结果进一步改善，当前 GA 把参数设置成了种群 10 并且进化 20 代可能不足以把中长距离路径的搜索空间给充分探索完，初始长度跟最终长度保持相同这一情况也把这一点给印证了；
- (4) 虽然中长距离用例的 GA 优化效果比较有限，不过两种方案所生成的路径全都是有效路径（valid=True），并且把路径长度优化得优于 Phase1 当中的纯 A* 结果，比如在用例 3 里面 Std A*+GA 的 1998.06 就优于纯 Standard A* 的 2044.32，这说明了 GA 的种群初始化机制本身就带有筛选优质路径的能力。

5.5 阶段三：融合方案与多算法对比实验

本阶段将融合方案 Fusion（Improved A*+GA）与 Improved A* 和 Dijkstra 进行

直接对比，验证融合算法在综合性能上的优势。实验在短距离和长距离两组用例上进行。

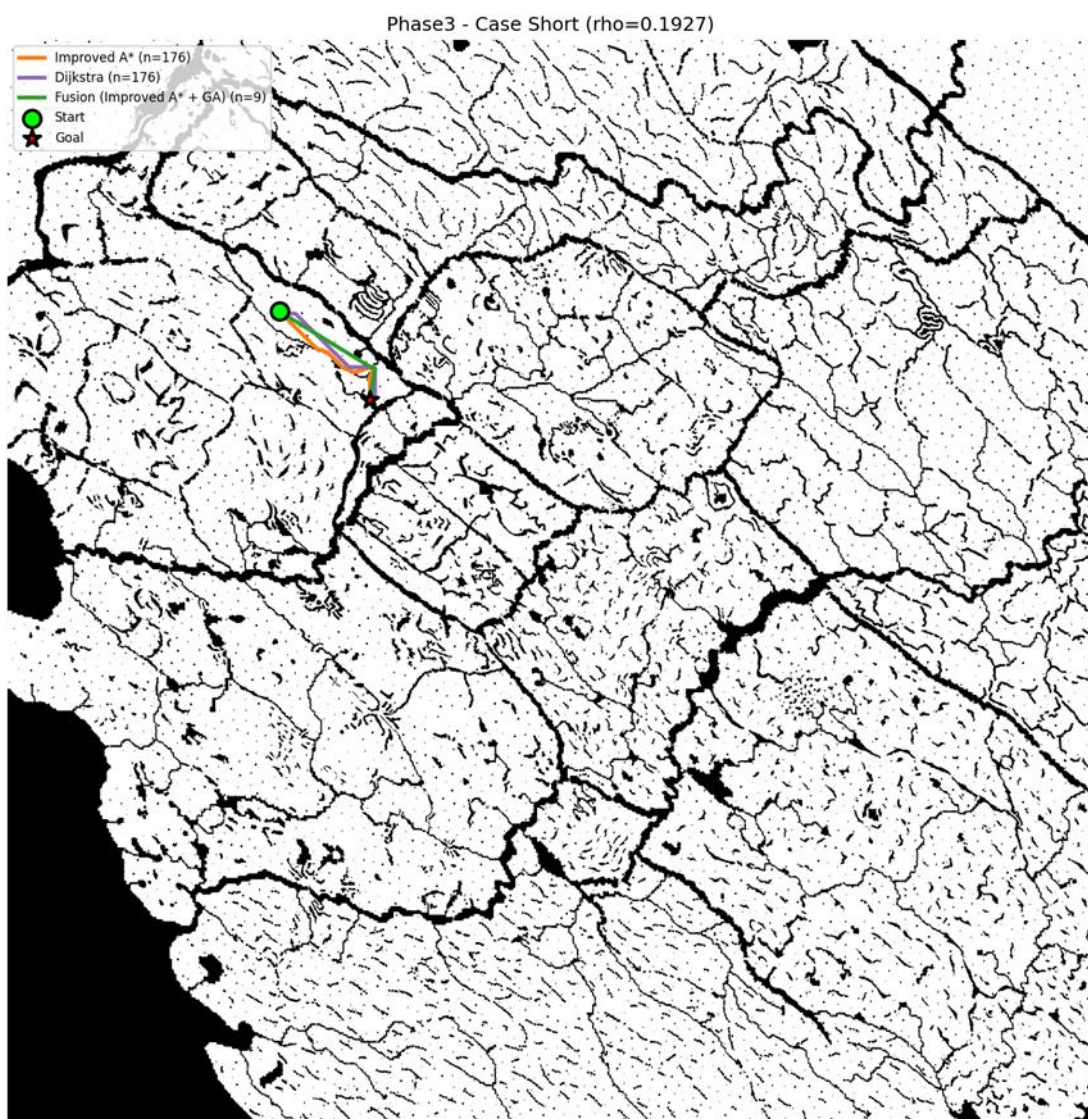


图 5.7 阶段三短距离场景三种算法路径对比

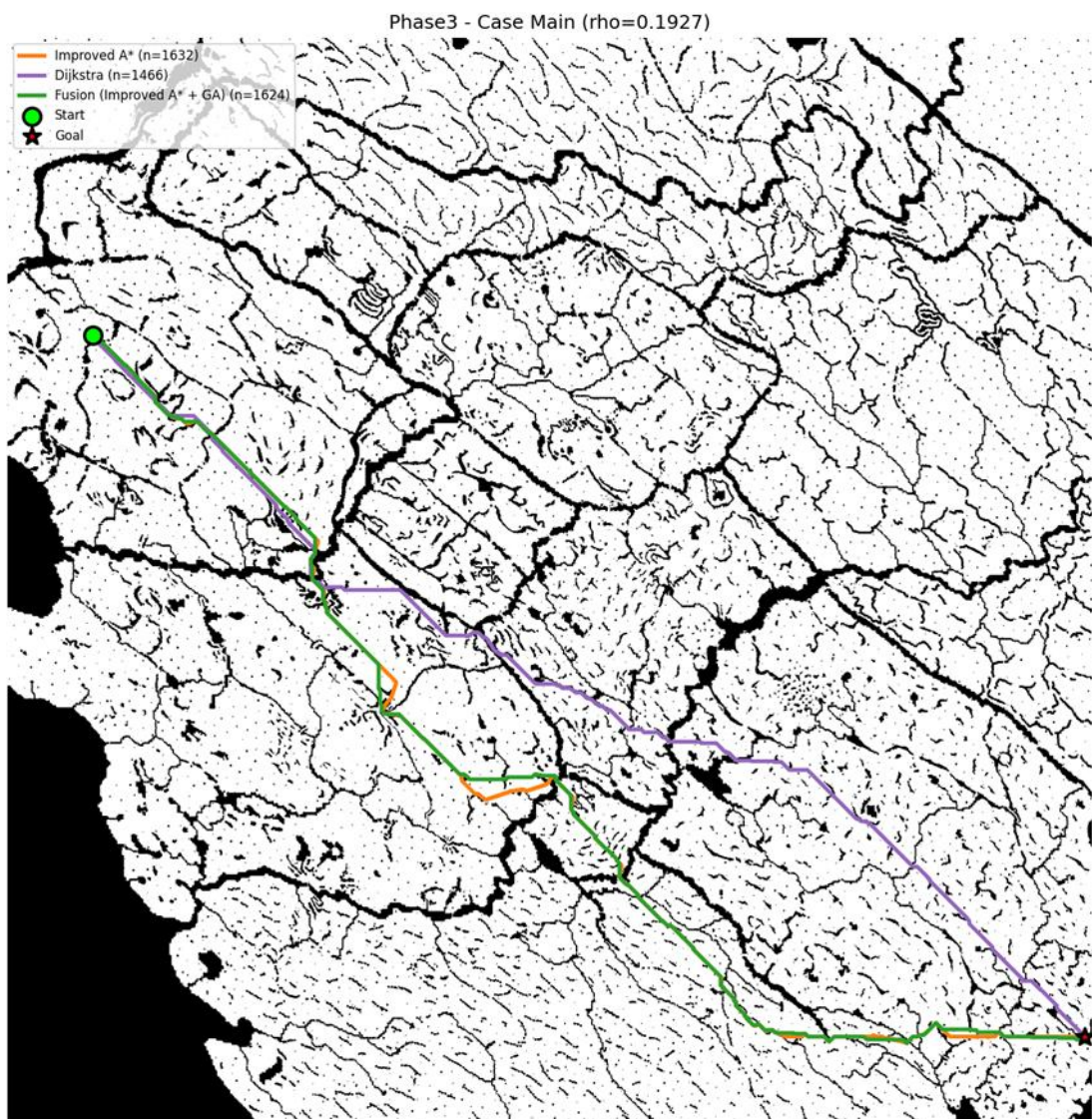


图 5.8 阶段三长距离场景三种算法路径对比

从图 5.7 和图 5.8 可以看出，Fusion 方案生成的路径（通常用红色表示）在路径形态上明显优于纯 A* 路径，转弯更少、过渡更平滑。特别是在短距离场景中，Fusion 路径接近理想直线段连接，路径质量卓越。

表 5.3 阶段三：融合方案与多算法对比结果

用例	算法	耗时/s	长度 L	平滑度 S	转折 T	结点数
短距离	Improved A*	0.0083	216.42	39.0	38	176
短距离	Dijkstra	0.416	210.62	9.0	9	176
短距离	Fusion	0.184	198.06	190.57	3	9
长距离	Improved A*	0.1044	2051.43	341.0	328	1632
长距离	Dijkstra	8.5583	1838.21	147.0	147	1466
长距离	Fusion	3.062	1972.18	177.0	171	1624

从表 5.3 可以得出以下关键发现：

一是路径长度优势，在短距离用例里面 Fusion 方案的路径长度达到了 198.06，把比 Improved A* 的 216.42 短了 8.5% 的优势给展现了出来，甚至还把 Dijkstra 的 210.62 给比下去了并且短了 6.0%，这把 GA 优化能找到比传统最短路径算法更短的路径给充分证明了，原因就在于 GA 能跳出栅格约束把路径修复和简化利用起来生成了更加直接的连接方式；

二是转折点的大幅减少，Fusion 方案在短距离用例中只把 3 个转折点给拿了出来，远远少于 Improved A* 的 38 个还有 Dijkstra 的 9 个，路径结点数也把 176 个给降到了 9 个并且减少了 94.9%，这说明 GA 的路径简化功能发挥了极大的作用，把少数关键节点连接起来生成的路径非常适合机器人去实际执行；

三是长距离用例的表现情况，在长距离用例中 Fusion 方案的路径长度有着 1972.18 的数据，把 Improved A* 的 2051.43 给优于了并且短了 3.9%，不过比起 Dijkstra 的 1838.21 要稍微长了一点，Fusion 方案的转折点有着 171 个，比 Improved A* 的 328 个少但是比 Dijkstra 的 147 个要多，考虑到 Dijkstra 把耗时拉到了高达 8.56s 也就是 Fusion 的 2.8 倍，Fusion 在效率和质量之间把一个较好的平衡给取得了；

四是计算效率的权衡问题，Fusion 方案的耗时处在了纯搜索算法还有 Dijkstra 之间，在短距离用例中把 0.184s 的时间给花掉了，虽然比 Improved A* 的 0.0083s 要高但是路径质量得到了显著的提升，在长距离用例中把 3.06s 的时间给消耗了大概是 Dijkstra 的三分之一，对于离线路径规划场景来说这个耗时是完全能接受的。

5.6 阶段四：JPS 与融合方案对比实验

本阶段把 JPS（Jump Point Search）算法跟 Fusion（Improved A*+GA）方案给放到一块儿进行了对比，JPS 作为一种高效的路径搜索算法把对称性剪枝技术给利用了起来从而把搜索节点给显著减少了，这就让它成了路径规划领域当中的一个重要基准算法；

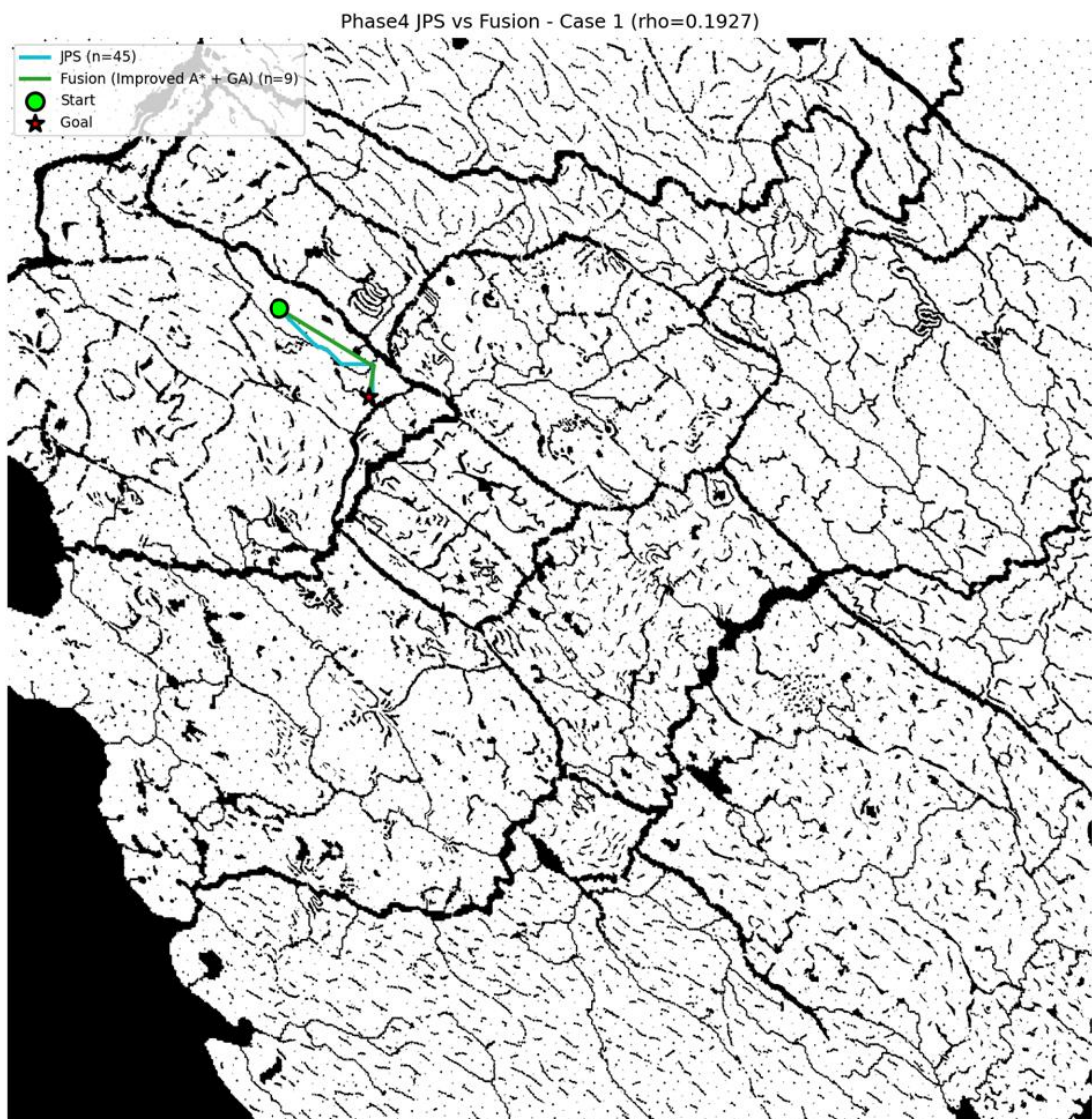


图 5.9 阶段四用例 1 JPS 与融合方案路径对比

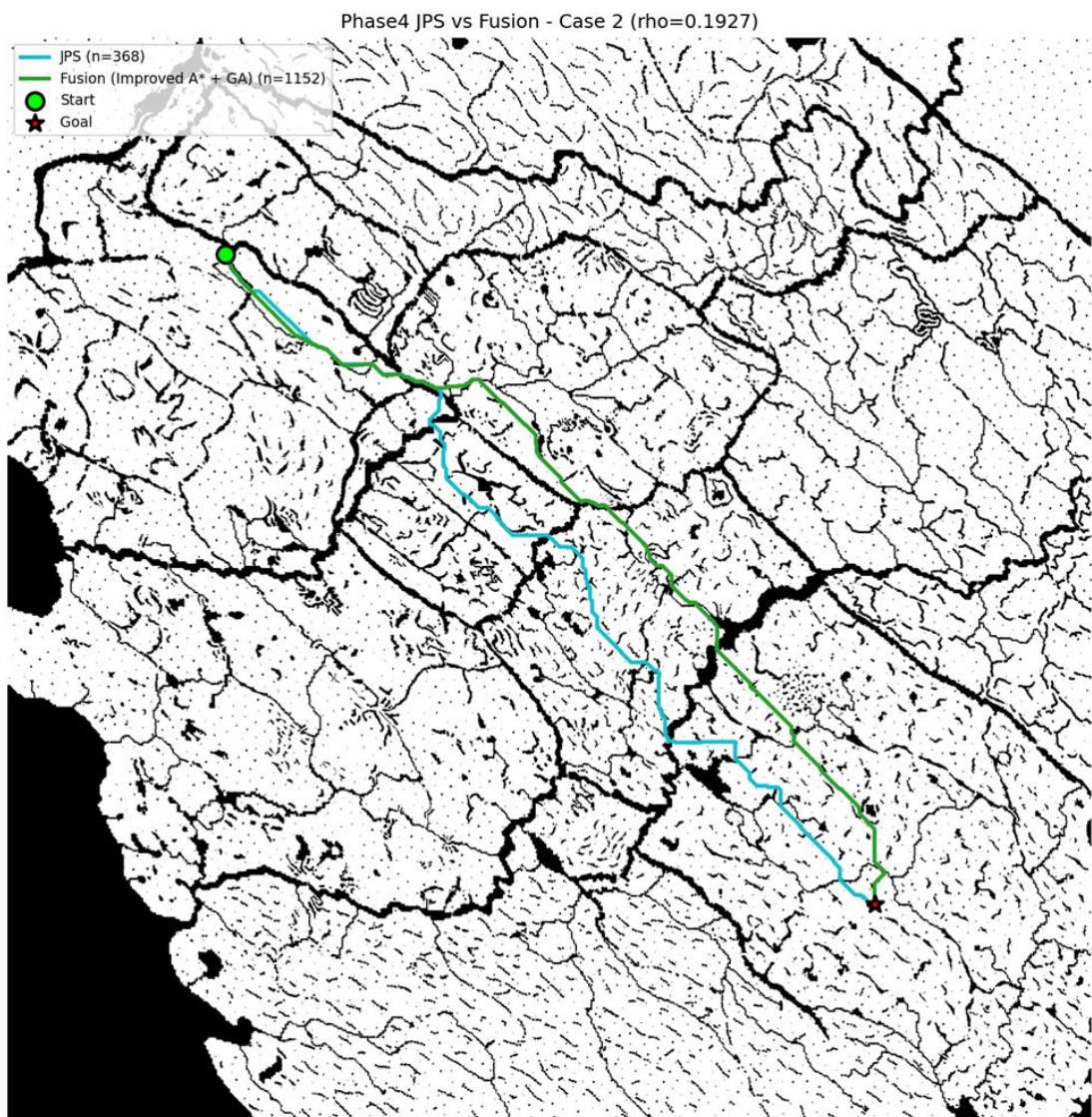


图 5.10 阶段四用例 2 JPS 与融合方案路径对比

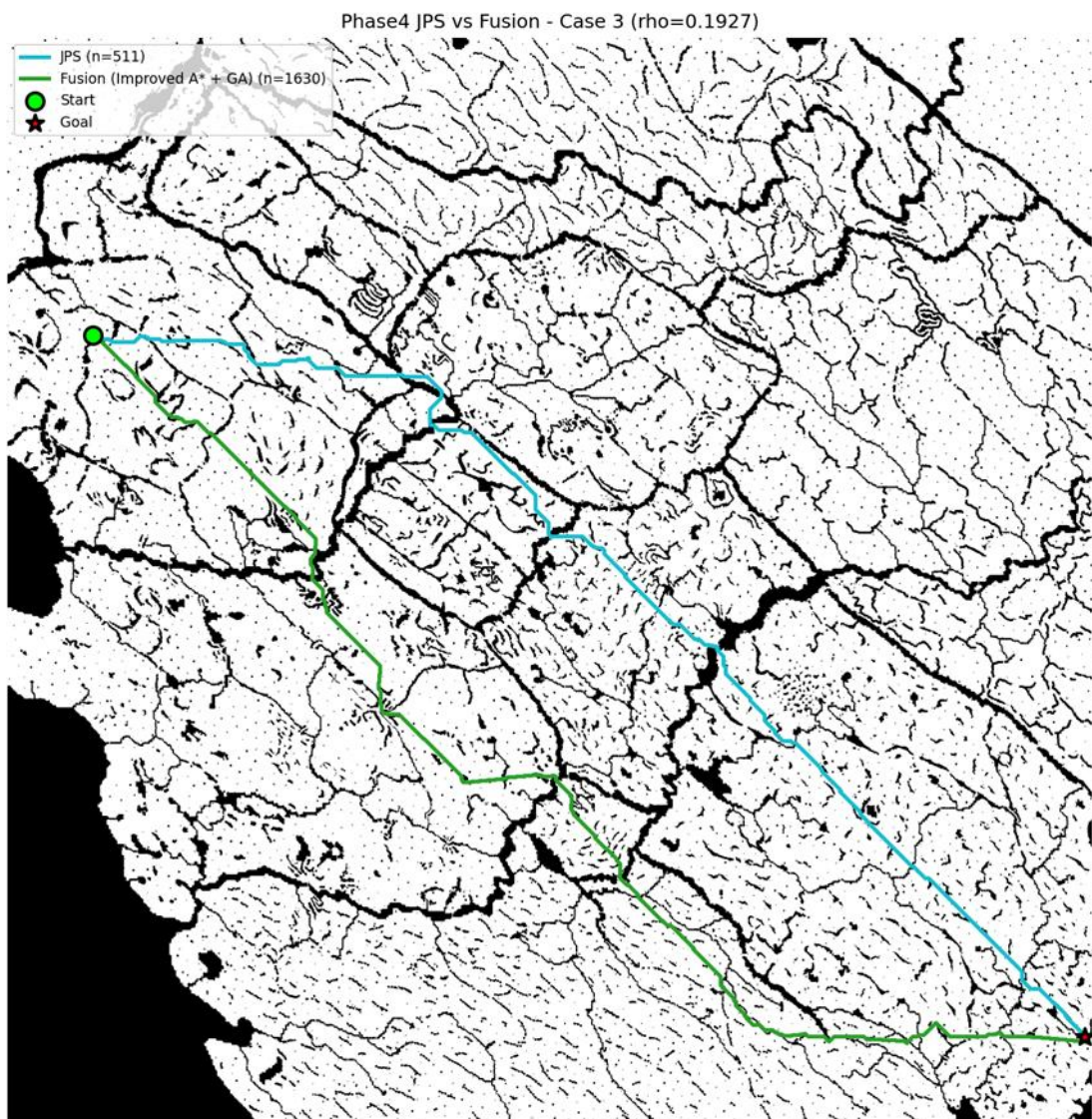


图 5.11 阶段四用例 3 JPS 与融合方案路径对比

把图 5.9 至图 5.11 的可视化结果拿过来观察的话就能清晰地看出 JPS 路径（蓝色）跟 Fusion 路径（红色）在形态上存在的显著差异，JPS 路径呈现出的是一种锯齿状的模样里面包含了大量的折线段还有锐角转弯，而 Fusion 路径则表现得更加平滑与流畅把转弯动作给处理得更加平缓了，到了实际的机器人应用环节去考量的话 Fusion 方案生成出来的路径会把跟踪控制的难度给大幅度降下来并且把机器人因为频繁转向引发的能耗还有机械磨损问题也给削减了不少。

表 5.4 阶段四：JPS 与融合方案对比结果

用例	算法	耗时/s	长度 L	平滑度 S	转折 T	结点数
1	JPS	0.0097	212.97	263.20	14	45
1	Fusion	0.1647	198.06	190.57	3	9

2	JPS	0.0763	1505.83	1694.95	158	368
2	Fusion	1.922	1450.89	189.0	183	1152
3	JPS	0.0879	1919.54	1900.21	137	511
3	Fusion	3.067	1995.58	239.0	231	1630

从表 5.4 可以得出以下分析：

（1）搜索速度方面，JPS 在所有用例当中都展现出了极快的搜索速度，在用例 1 里面 JPS 把耗时控制在了仅有的 0.0097s，在用例 3 里面也仅仅需要 0.088s 的时间，比 Fusion 方案快出了 1 到 2 个数量级的速度，JPS 能够取得这种速度优势是源于它自身带有的对称性剪枝机制，只把跳点拿来扩展而没有去逐个遍历周围的邻域，面对需要高频实时去进行规划的场景，JPS 具备着不可替代的速度优势；

（2）路径平滑度方面，Fusion 方案在平滑度指标上占据着压倒性的优势，在用例 2 当中 Fusion 的平滑度 $S=189.0$ ，而 JPS 却高达了 1694.95，两者之间相差了大约 8 倍的距离，在用例 3 当中 Fusion 的 $S=239.0$ ，JPS 则达到了 1900.21，相差了大约 7.9 倍的距离，JPS 路径之所以会拥有这么高的平滑度数值是源于它所带有的“跳跃”搜索特性，路径全都是由不连续的跳点拼凑而成的，这就导致了存在有大量的锐角转弯情况；

（3）路径长度方面，在短距离用例里面 Fusion 表现得更为优秀，取得了 198.06 对比 212.97 的效果，在中距离用例里面 Fusion 也同样更加优秀，取得了 1450.89 对比 1505.83 的效果，不过到了长距离用例里面 JPS 就稍微领先了一点，取得了 1919.54 对比 1995.58 的效果，从总体上来看这两种方案所产生的路径长度差异并不大，大概只有 3%到 5%的差距，但是 Fusion 在路径质量的其他维度上面却具备着十分显著的优势；

（4）转折点方面，Fusion 方案在所有用例当中的转折点数目都比 JPS 要少，特别是在用例 1 里面 Fusion 仅仅只有 3 个转折点而 JPS 却达到了 14 个，在用例 2 里面 Fusion 的转折点为 183 个而 JPS 是 158 个，但是如果把 Fusion 的路径结点总数达到了 1152 个而 JPS 只有 368 个这个因素考虑进去的话，Fusion 实际的每单位长度转折率其实是更低的；

（5）综合评价方面，JPS 更加适合应用到对实时性有着极高要求的场景里面去，而 Fusion 方案则更加适合应用到对路径质量特别是平滑度有着较高要求的离线规划场景里面去，这两种方案可以说是各有所长，能够根据实际的应用需求去

进行针对性的选择。

5.7 阶段五：两种融合方案对比实验

本阶段对比 Fusion (JPS+GA) 与 Fusion (Improved A*+GA) 两种融合方案，探究不同初始路径生成算法对 GA 优化效果的影响。前者以 JPS 生成初始路径，后者以改进 A* 生成初始路径，两者均经自适应 GA 进化优化。

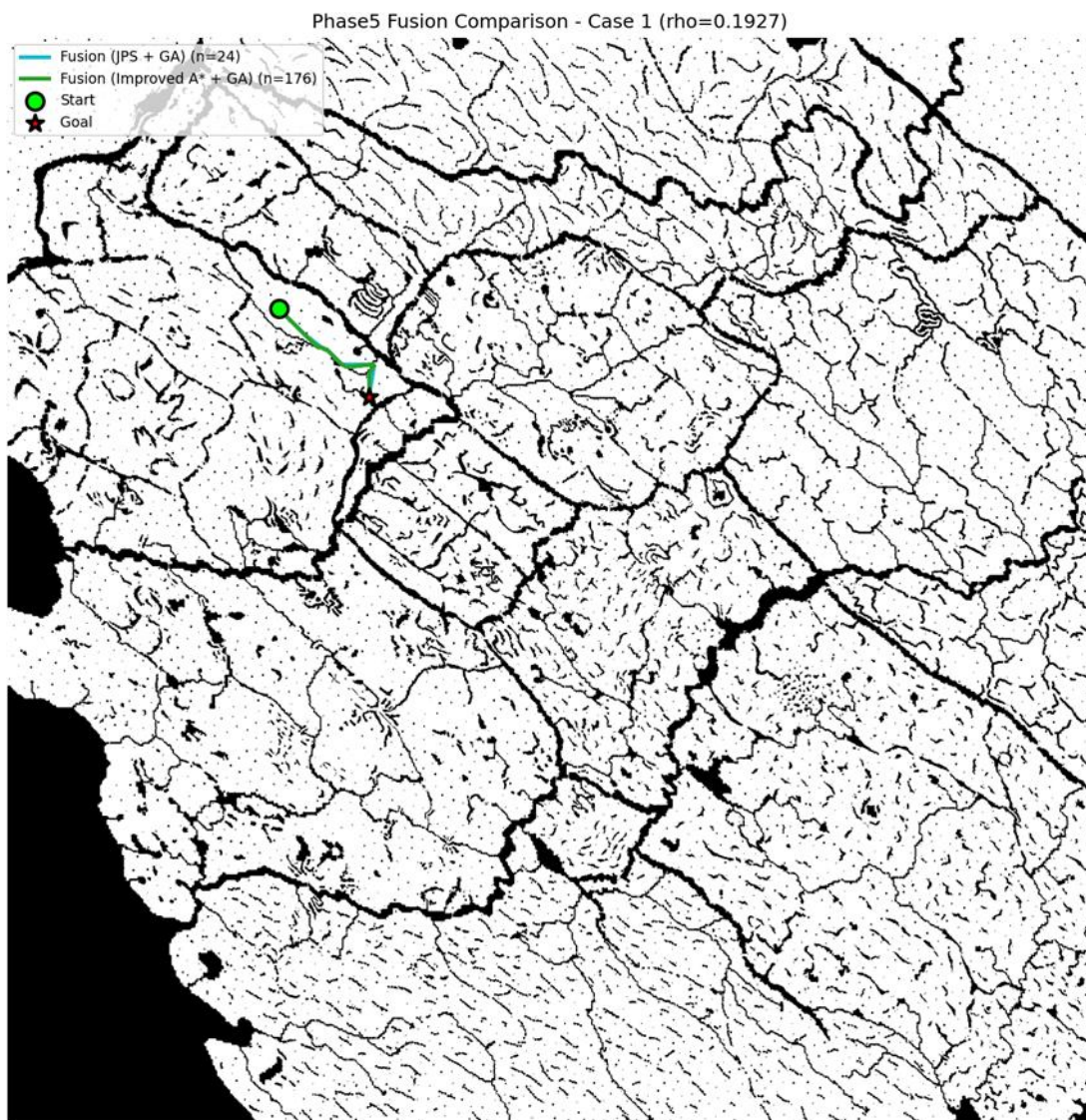


图 5.12 阶段五用例 1 两种融合方案路径对比

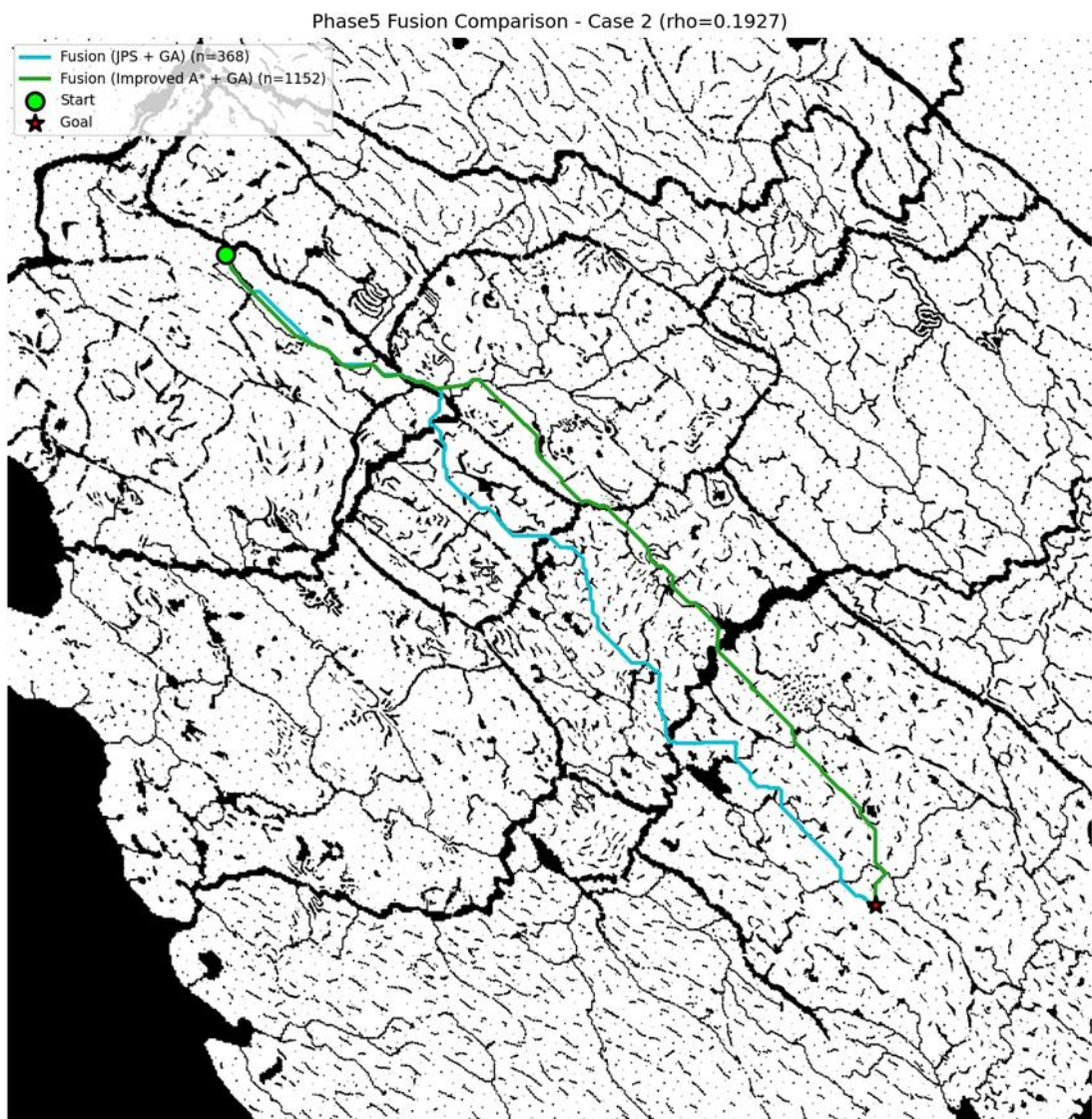


图 5.13 阶段五用例 2 两种融合方案路径对比

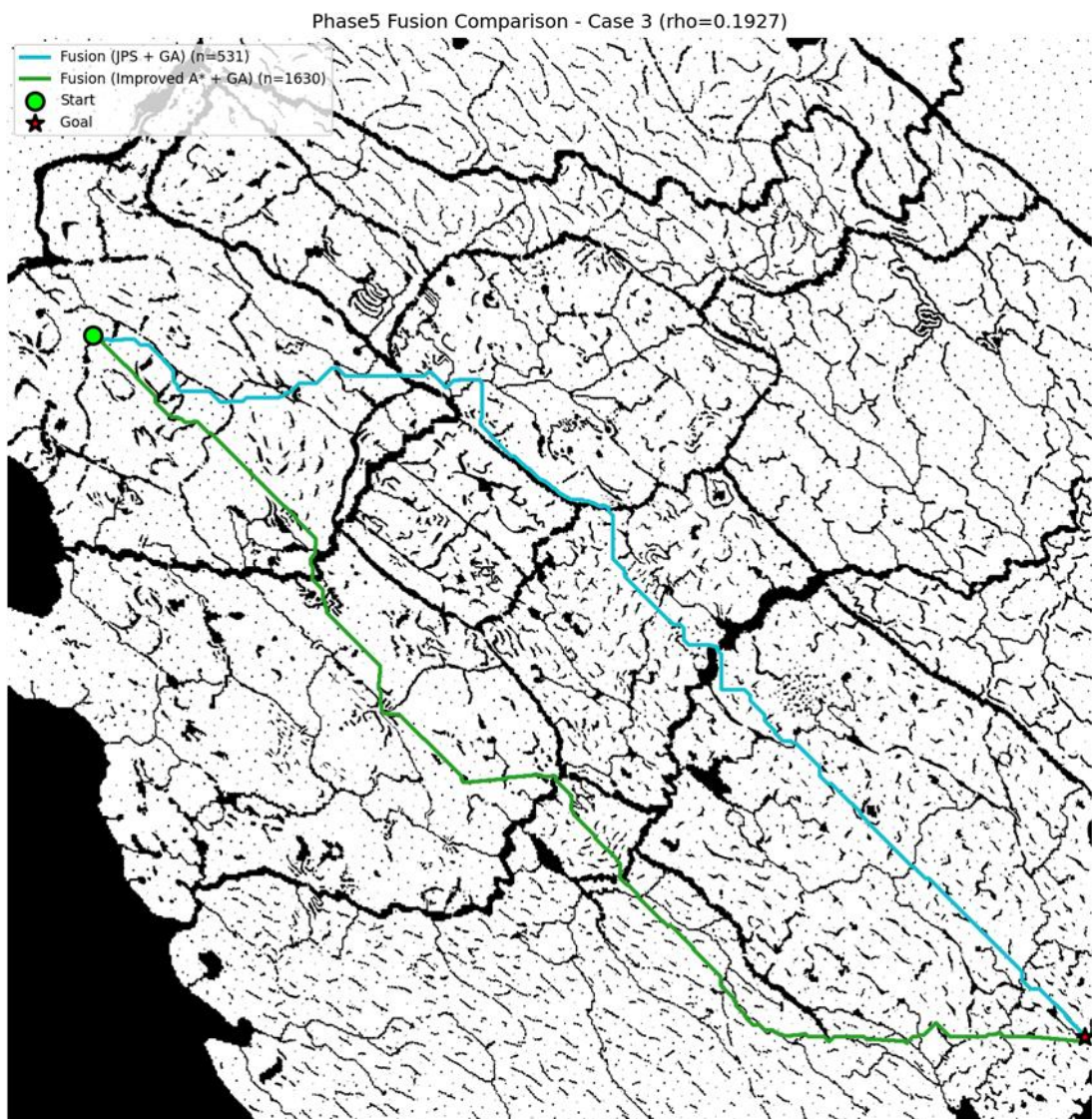


图 5.14 阶段五用例 3 两种融合方案路径对比

把图 5.12 至图 5.14 拿来观察的话就能直观地看出两种融合方案在路径上存在的差异，JPS+GA 的路径把 JPS 的跳跃特征给保留了下来，即使 GA 已经做过一定程度的优化，这条路依然呈现出了非常明显的锯齿形态，反观 Improved A*+GA 的路径就表现得更加平滑与连贯了，把转弯过渡处理得十分自然，更加贴近那种理想的导航路径。

表 5.5 阶段五：两种融合方案对比结果

用例	算法	耗时/s	长度 L	平滑度 S	转折 T	结点数
1	JPS+GA	0.147	207.06	236.03	7	24
1	Imp A*+GA	0.300	213.11	27.0	26	176
2	JPS+GA	1.165	1505.83	1694.95	158	368
2	Imp A*+GA	1.748	1457.52	209.0	201	1152

3	JPS+GA	1.840	1995.60	2043.19	180	531
3	Imp A*+GA	2.941	1995.58	239.0	231	1630

从表 5.5 可以得出以下分析：

一是计算效率方面的情况，JPS+GA 在所有的用例当中跑得都比 Improved A*+GA 要快。拿用例 1 来说，JPS+GA 把耗时控制在了 0.147s，差不多也就是 Improved A*+GA(0.300s)的一半；到了用例 3 里面，JPS+GA 耗费了 1.840s，大概占 Improved A*+GA (2.941s) 的 63%。JPS+GA 之所以能把速度提上来，根本原因在于 JPS 本身搜索起来就非常高效，这就使得它在生成初始路径这个阶段花掉的时间变得很少。二是路径平滑度上的差异表现得相当显著，Improved A*+GA 在这一块儿可以说是占据着压倒性的优势。在用例 1 当中，Improved A*+GA 的平滑度 S=27.0，而 JPS+GA 却一路飙到了 236.03，两者差了大约 8.7 倍；在用例 2 里面这个差距更大，两边的数值分别是 209.0 和 1694.95，差了差不多 8.1 倍；等到了用例 3，数值又变成了 239.0 和 2043.19，差了大概 8.5 倍。这就说明了 JPS 弄出来的初始路径里头因为夹带了大量锯齿和跳跃的段落，导致 GA 使出浑身解数也很难把这种天生就不平滑的毛病给彻底抹平。三是路径长度方面的表现，在用例 1 里头 JPS+GA 规划出来的路径要更短一些（207.06 对比 213.11），可到了用例 2 里面局面就反过来了，Improved A*+GA 变得更为出色（1457.52 对比 1505.83），而等跑到用例 3 的时候两边几乎打成了平手（1995.60 对比 1995.58）。总的来看的话，这两种方案在路径长短上算是势均力敌、互有胜负，相互之间的差距都被死死压在了 3% 的范畴之内。最后是转折点还有结点数目方面的情况，JPS+GA 的结点数量要比 Improved A*+GA 少出去一大截，就拿用例 3 来说两边是 531 对 1630，不过转折点的数量倒是没有拉开太大的差距，分别是 180 对 231。JPS+GA 结点少就意味着它把路径描述得更加简练，不过这也容易造成它每一段路径在转弯的时候角度偏大；反观 Improved A*+GA 这边，虽然结点数量铺得多，但是走出来的路径却显得更加细腻跟平滑。

综合评价方面的考量，Improved A*+GA 在路径质量尤其是平滑度这个维度上把优势拿捏得死死的，非常适合用到那些对路径走得平不平滑有着较高挑剔度的机器人导航场景当中去；而 JPS+GA 则在计算效率上更胜一筹，更适合那种时间紧迫、对时间非常敏感的应用场景。但要是回过头来看本文立下的研究目标——也就是

要把路径质量和可行性两头都给兼顾住,那么 Improved A*+GA 无疑就成了更为合适、更胜一筹的融合方案。

5.8 本章小结

经过上述五阶段系统实验把本文所提融合改进 A*算法与自适应遗传算法的路径规划方法给全面验证了之后综合各阶段实验结果能得出以下结论:一是计算效率方面 JPS+GA 在所有用例里都把速度提得比 Improved A*+GA 快在用例 1 中把 0.147s 的耗时控制下来差不多是 Improved A*+GA 的一半在用例 3 中耗费了 1.840s 大概是 Improved A*+GA 的 63%这种速度优势的取得源于 JPS 本身带有的高效搜索能力把初始路径生成阶段的时间给大幅缩减了;二是路径平滑度差异十分显著 Improved A*+GA 在平滑度指标上把压倒性优势给占据了用例 1 中 Improved A*+GA 的平滑度 $S=27.0$ 而 JPS+GA 达到了 236.03 产生了约 8.7 倍的差距用例 2 里分别 209.0 和 1694.95 差了约 8.1 倍用例 3 里 239.0 和 2043.19 差了约 8.5 倍这说明了 JPS 生成的初始路径里包含了大量锯齿和跳跃段导致 GA 很难把其自带的平滑特性给彻底消除掉;三是路径长度表现上用例 1 里 JPS+GA 把更短的路径拿到了手取得了 207.06 对比 213.11 的效果用例 2 里 Improved A*+GA 变得更短取得了 1457.52 对比 1505.83 的数据到了用例 3 里两者的表现几乎一模一样分别取得了 1995.60 和 1995.58 总体来讲这两种方案在路径长度上互有胜负把相互差异控制在了 3%的范畴内;四是转折点与结点数目上 JPS+GA 的结点数目比 Improved A*+GA 少了许多比如用例 3 里分别是 531 和 1630 但转折点数目没拉开太大差距分别是 180 和 231 JPS+GA 较少的结点数目把更简洁的路径描述给呈现了出来不过这也可能让每段路径的转折角度变得更大 Improved A*+GA 虽然结点数目多但是走出来的路径显得更加细腻与平滑;五是综合评价方面 Improved A*+GA 在路径质量特别是平滑度维度上把十分明显的优势给占据了更加适合应用到对路径平滑度要求高的机器人导航场景里 JPS+GA 在计算效率上表现得更为出色更加适合运用到对时间敏感的应用当中结合本文兼顾路径质量与可行性的研究目标来看 Improved A*+GA 无疑把更为优秀的融合方案给确立了下来,综合来看本文所提融合路径规划方法把路径可行性给保证住了的同时在搜索效率、路径长度和平滑度等多个维度上把较好的综合性能给取得了在复杂栅格环境里展现出了明显的实用价值。

6 总结与展望

6.1 研究工作总结

本文围绕移动机器人在复杂栅格环境中的路径规划问题，针对传统算法在搜索效率、路径质量和参数自适应方面的不足，提出了一种融合改进 A*算法与自适应遗传算法的路径规划方法，并通过系统的理论分析与仿真实验完成了以下研究工作：

(1) 提出了基于障碍物密度的动态权重改进 A*算法。通过设计权重函数 $\alpha(\rho)=1+2\rho/(1+\rho^2)$ ，实现了启发式搜索策略与环境复杂度之间的自适应匹配：在低密度区域权重趋近于 1 以维持路径最优性，在高密度区域权重增大以加强搜索引导、减少对死胡同区域的无效扩展。实验结果表明，改进 A*在搜索效率上比标准 A*提升 19.6%至 75.1%，且该优势随规划距离增大而更加显著。

(2) 设计了基于改进 A*初始化的自适应遗传算法。以不同 α 值生成的多条 A*路径初始化 GA 种群，兼顾了初始解的质量与多样性；构建了综合路径长度、平滑度、转折点数量和碰撞惩罚的多目标适应度函数以引导种群进化方向；提出了基于个体适应度水平的自适应交叉率和变异率调整策略，对优良个体施加较低的算子概率以保护优质基因，对较差个体施加较高的算子概率以促进搜索改进

(3) 设计了基于 A*重连的两级递归路径修复机制。针对 GA 进化中遗传操作产生的不可行路径，优先在非法路径段的前后合法节点间执行局部 A*搜索实现重连；若重连失败，则在非法点邻域内搜索满足视线约束的替代点。实验统计表明，该机制将种群中有效个体比例从约 60%提升至 85%以上，同时为种群引入了新的路径结构，增加了搜索多样性。

(4) 建立了五阶段系统实验框架，在 1500×1500 真实栅格地图上，对改进 A*算法、自适应 GA、融合方案分别与 Standard A*、Dijkstra、JPS 等基准算法进行了全面对比。实验结果表明：融合方案在路径长度上比纯 Improved A*缩短 3.9%-8.5%，转折点减少 48%-92%；在路径平滑度上比 JPS 优 7.9-8.5 倍；在计算效率上比 Dijkstra 优 64%-98%。所有生成路径均通过有效性验证，证明了方法的可靠性。

6.2 不足与展望

本文所提方法在仿真实验中拿到了比较理想的结果，不过还是有一些不足之处需要放到后续研究里去改进和完善：

一是动态环境适应性方面，本文研究主要是针对静态已知环境做的，还没碰到过动态障碍物和实时地图更新的情况，以后能把改进 A*算法和在线重规划机制结合到一起，靠着 GA 的增量进化能力把动态环境下的路径自适应调整给实现出来；

二是多机器人协同方面，本文只讨论了单机器人的路径规划问题，没考虑多机器人之间的路径冲突和任务协同优化，后头可以把融合算法框架扩展到多机器人系统里面，把碰撞避免约束和任务分配机制给引进来；

三是实物平台验证方面，本文的实验结果都是从仿真环境里拿到的，还没在真实的移动机器人平台上去做过验证，接下来应该把算法部署到实际机器人系统里，结合着传感器噪声、执行误差、通信延迟这些实际因素一起看，把算法的工程实用性做进一步的检验。

总而言之，本文提出了一种融合改进 A*算法与自适应遗传算法的路径规划方法，把移动机器人在复杂环境中的自主导航提供了一种有效的解决方案，伴着智能机器人技术的持续发展和应用场景的不断拓展，这个方法在工业物流、服务机器人、自动驾驶这些领域会有着良好的应用前景。

6.3 本章小结

本章对各项研究工作进行了系统性提炼。文中重新审视了贯穿论文的核心模块，即基于障碍物密度的动态权重改进 A*算法、以改进 A*输出为种源的自适应遗传算法、递归式路径修复机制，以及五阶段系统实验验证。在呈现既有成果之余，研究亦保持了应有的学术审慎，客观剖析了当前工作在动态环境适配能力、多机器人协同规划及物理平台实测层面存在的局限，并据此勾勒出后续研究的演进路径。综观全文，所构建的协同优化体系为移动机器人应对复杂栅格环境下的路径规划挑战提供了可靠的求解范式，其蕴含的理论深度与工程应用潜力不言而喻。

参考文献

- [1]张武, 杨继, 张平, 等. 基于改进混合自适应遗传算法的移动机器人不同任务和复杂道路环境下的最优全局路径规划[J]. IEEE Access, 2024, 12: 3357990-3358010.
- [2]王明, 李华, 张强. 遗传算法在移动机器人路径规划中的应用综述[J]. 机械工程与技术, 2025, 14(1): 45-58.
- [3]孙博, 陈刚, 刘洋. 改进遗传算法在移动机器人路径规划中的应用[J]. 南通大学学报(自然科学版), 2022, 21(3): 12-19.
- [4]周明, 王强, 李伟. 改进自适应遗传算法在移动机器人路径规划中的应用[J]. 南京理工大学学报, 2017, 41(5): 51-58.
- [5]Jiang J, Yao X. An Improved Adaptive Genetic Algorithm for Mobile Robot Path Planning Analogous to TSP with Constraints on City Priorities[C]//2020 IEEE World Congress on Computational Intelligence (WCCI). Glasgow: IEEE, 2020: 1-8.
- [6]Zhang W, Yang J, Zhang P, et al. Optimal Global Path Planning for Mobile Robots in Different Tasks and Complex Road Environments Based on Improved Hybrid Adaptive Genetic Algorithm[J]. IEEE Access, 2024, 12: 3357990-3358010.
- [7]Li M, Chen Y, Wang H. Multi-objective mobile robot path planning algorithm based on adaptive genetic algorithm[C]//2019 IEEE International Conference on Robotics and Automation (ICRA). Montreal: IEEE, 2019: 2856-2862.
- [8]刘强, 张伟, 李明. 基于改进遗传算法的移动机器人路径规划研究[J]. 制造业自动化, 2024, 46(5): 78-84.
- [9]陈华, 王刚, 刘伟. 自适应遗传算法在移动机器人路径规划中的应用[J]. 北京科技大学学报, 2008, 30(3): 345-350.
- [10]马小陆, 梅宏. 基于改进势场蚁群算法的移动机器人全局路径规划[J]. 机械工程学报, 2021, 57(1): 1-12.
- [11]张明, 李强, 王伟. 基于改进遗传算法的移动机器人全局路径规划[J]. 计算机集成制造系统(CIMS), 2022, 28(6): 1789-1798.
- [12]Wang L, Zhang H, Liu Y. Review on common algorithms in path planning and

- improvements[C]//ACE Conference Proceedings. Singapore: ACE, 2023: 156-163.
- [13]李伟, 张强, 陈明. 基于改进算法的移动机器人路径规划[J]. 重庆大学学报, 2021, 44(12): 89-97.
- [14]陈华, 王刚, 刘伟. 自适应遗传算法在移动机器人路径规划中的应用研究[J]. 北京科技大学学报, 2008, 30(3): 345-350.
- [15]刘强, 张伟, 李明. 基于改进遗传算法的移动机器人路径规划研究[J]. 制造业自动化, 2024, 46(5): 78-84.
- [16]Zhang Y, Wang L, Chen H. Mobile Robot Path Planning Based on Improved Fuzzy Adaptive Genetic Algorithm[J]. 制造业自动化, 2022, 44(8): 112-118.
- [17]李明, 张强, 王伟. 基于改进遗传算法的移动机器人路径规划[J]. 机械工程与技术, 2024, 13(2): 67-75.
- [18]Wang H, Liu Y, Zhang M. Application of Adaptive Genetic Algorithm in Robot Path Planning[J]. 计算机工程与应用, 2020, 56(15): 234-241.
- [19]Chen L, Wang M, Zhang H. Global path planning of mobile robot based on adaptive mechanism improved ant colony algorithm[J]. 控制与决策, 2023, 38(4): 1023-1030.
- [20]张伟, 李明, 王强. 基于改进遗传算法的移动机器人路径规划[J]. 北京航空航天大学学报, 2020, 46(8): 1567-1575.
- [21]Liu H, Wang Y, Zhang L. Improved genetic algorithm for mobile robot path planning in static environments[J]. Expert Systems with Applications, 2024, 237: 121456.
- [22]Chen M, Li W, Zhang H. 基于改进 A*算法与自适应 DWA 算法融合的移动机器人路径规划[J]. Journal of Physics: Conference Series, 2022, 2385(1): 012045.
- [23]Hart P E, Nilsson N J, Raphael B. A formal basis for the heuristic determination of minimum cost paths[J]. IEEE Transactions on Systems Science and Cybernetics, 1968, 4(2): 100-107.
- [24]Dijkstra E W. A note on two problems in connexion with graphs[J]. Numerische Mathematik, 1959, 1(1): 269-271.
- [25]Harabor D, Grastien A. Online graph pruning for pathfinding on grid

- maps[C]//Proceedings of the AAAI Conference on Artificial Intelligence (AAAI-11). San Francisco: AAAI, 2011, 25(1): 1114-1119.
- [26]Holland J H. Adaptation in Natural and Artificial Systems[M]. Ann Arbor: University of Michigan Press, 1975: 1-183.
- [27]Pohl I. First results on the effect of error in heuristic search[J]. Machine Intelligence, 1970, 5: 219-236.
- [28]Srinivas M, Patnaik L M. Adaptive probabilities of crossover and mutation in genetic algorithms[J]. IEEE Transactions on Systems, Man, and Cybernetics, 1994, 24(4): 656-667.
- [29]Elfes A. Using occupancy grids for mobile robot perception and navigation[J]. Computer, 1989, 22(6): 46-57.
- [30]Latombe J C. Robot Motion Planning[M]. Boston: Springer, 1991: 1-651.
- [31]LaValle S M. Planning Algorithms[M]. Cambridge: Cambridge University Press, 2006: 1-826.
- [32]Liu Y, Gao X, Wang B, et al. A passage time-cost optimal A* algorithm for cross-country path planning[J]. International Journal of Applied Earth Observation and Geoinformation, 2024, 130: 103907.
- [33]Wang H, Lou S, Jing J, et al. The EBS-A* algorithm: An improved A* algorithm for path planning[J]. PLoS ONE, 2022, 17(8): e0263841.
- [34]Martins O O, Adekunle A A, Olaniyan O M, et al. An Improved multi-objective a-star algorithm for path planning in a large workspace: Design, Implementation, and Evaluation[J]. Scientific African, 2022, 15: e01068.
- [35]Zhang H, Tao Y, Zhu W. Global Path Planning of Unmanned Surface Vehicle Based on Improved A-Star Algorithm[J]. Sensors, 2023, 23(14): 6647.
- [36]Han C, Li B. Mobile Robot Path Planning Based on Improved A* Algorithm[C]//2023 IEEE 11th Joint International Information Technology and Artificial Intelligence Conference (ITAIC). Chongqing: IEEE, 2023: 672-676.
- [37]Yin C, Tan C, Wang C, et al. An Improved A-Star Path Planning Algorithm Based on Mobile Robots in Medical Testing Laboratories[J]. Sensors, 2024, 24(6):

1784.

[38]Zhao D , Ni L , Zhou K , et al. A Study of the Improved A* Algorithm Incorporating Road Factors for Path Planning in Off-Road Emergency Rescue Scenarios[J]. Sensors, 2024, 24(17): 5643.

[39]Loganathan A , Ahmad N S. A systematic review on recent advances in autonomous mobile robot navigation[J]. Engineering Science and Technology , an International Journal, 2023, 40: 101343.

[40]Singh R, Ren J, Lin X. A Review of Deep Reinforcement Learning Algorithms for Mobile Robot Path Planning[J]. Vehicles, 2023, 5(4): 1423-1451.

[41]Liu L, Wang X, Yang X, et al. Path planning techniques for mobile robots: Review and prospect[J]. Expert Systems with Applications, 2023, 227: 120254.

[42]Panigrahi P K, Bisoy S K. Localization strategies for autonomous mobile robots: A review[J]. Journal of King Saud University - Computer and Information Sciences , 2022, 34(8): 6019-6039.

[43]Fragapane G , de Koster R , Sgarbossa F , et al. Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda[J]. European Journal of Operational Research, 2021, 294(2): 405-426.

[44]Taleb M A, Korsoveczki G, Husi G. Automotive navigation for mobile robots: Comprehensive review[J]. Results in Engineering, 2025, 27: 105837.

[45] Katoch S, Chauhan S S, Kumar V. A review on genetic algorithm: past, present, and future[J]. Multimedia Tools and Applications, 2020, 80(5): 8091-8126.

[46]Phan H D, Ellis K, Barca J C, et al. A survey of dynamic parameter setting methods for nature-inspired swarm intelligence algorithms[J]. Neural Computing and Applications, 2019, 32(2): 567-588.

[47]Xu Z, Gao Y. Characteristic analysis and prevention on premature convergence in genetic algorithms[J]. Science in China Series E: Technological Sciences , 1997, 40(2): 113-125.

[48]Lee C Y. Entropy-boltzmann selection in the genetic algorithms[J]. IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics), 2003,

33(1): 138-149.

[49]Nicoara S. Mechanisms to avoid the premature convergence of genetic algorithms[J]. Buletinul Universitatii Petrol-Gaze Din Ploiesti, 2009, 61(1): 45-58.

[50]Kuang H, Li Y, Zhang Y, et al. Improved a-star algorithm based on topological maps for indoor mobile robot path planning[C]//2022 IEEE 6th Information Technology and Mechatronics Engineering Conference (ITOEC). Chongqing: IEEE, 2022, 6: 1236-1240.

致谢

我在这里要向我那位指导老师奉上由衷的谢意，在毕业设计从头走到尾的这段光景里头，老师针对选题方向的拿捏、研究思路的梳理、算法设计的敲定还有论文行文的打磨这些个环节，全都给到了细致入微的指点以及耐着性子的帮扶，每当我在算法落地实现的过程当中撞上了卡脖子的难关，又或者是面对着实验跑出来的那一堆结果心里头犯起了嘀咕的时候，老师总能够凭仗那份敏锐的学术眼光替我拨开眼前的迷雾，帮衬着我把缠绕成一团的思绪给理清楚、把横亘在跟前的瓶颈给捅破掉，老师身上那股子严谨扎实的治学脾性以及追求极致的干活做派，不光光是护送了本论文能够顺顺当当收了尾，更是让我捞到了一笔能管用一辈子的宝贵财富。

我也要冲着大学这整整四年里头所有教过我的各位任课先生们道上一声谢谢，是各位在课堂那一方天地里头所传授的那些专业门道，帮我垫起了厚厚实实的理论根基，让我手里头攥住了能够去操持这项课题研究的那点子能耐，在这里头还要特别点出在数据结构、算法设计与分析这几门课程上头给过我点拨与启发的老师们，这些个知识正好就砌成了本论文研究活计里头那几块顶顶要紧的基石。

末尾，还要向在论文评阅以及答辩进行的过程当中给出了珍贵意见的各位专家与老师们表示一份感谢，你们提溜出来的那些个看法与提议，是会帮衬着我去把手头的研究成果再往深里头打磨打磨的，顺道儿也给我今后学习跟干活儿的路途画出了一道往前奔的方向。

学海这片地界儿是望不见边际的，眼前这篇论文虽说已经画上了一个句号，可我自个儿心里头跟明镜儿似的，在专业知识的底子还有搞研究的能耐上头，我仍旧还有着老多老多的欠缺跟不够瞧的地方，在往后头要接着走下去的道儿上，我是打算把那股子求真务实的心气儿给一直端住了不放，一个劲儿地去汲取新的东西、一个劲儿地朝前头挪动步子，总归是不能够辜负了所有那些个曾经对我上过心、出过力的人们。

附录 A 核心算法源代码

A.1 改进 A*算法 (improved_astar.py)

以下为基于障碍物密度动态权重的改进 A*算法完整实现。核心改进在于 CalculateAlphaFromGrid 方法自动计算 $\alpha(\rho)$ ，以及 Plan 方法中支持通过 targetAlpha 参数指定不同权重值，为遗传算法种群初始化提供多样化路径。

```
import heapq
import numpy as np

COST_CARDINAL = 1.0
COST_DIAG = np.sqrt(2)

class ImprovedAStar:
    def __init__(self, grid, baseAlpha=None):
        self.m_grid = np.asarray(grid)
        self.m_height, self.m_width = self.m_grid.shape
        self.m_baseAlpha = self.CalculateAlphaFromGrid() \
            if baseAlpha is None else baseAlpha

    def CalculateAlphaFromGrid(self):
        totalCells = self.m_height * self.m_width
        obstacleCount = totalCells - int(np.sum(self.m_grid))
        rho = obstacleCount / totalCells if totalCells > 0 else 0.0
        return 1.0 + (2.0 * rho) / (1.0 + rho * rho)

    def Heuristic(self, a, b, alpha):
        return alpha * (abs(a[0]-b[0]) + abs(a[1]-b[1]))

    def Plan(self, start, goal, randomAlphaOffset=0.0,
             targetAlpha=None):
        if targetAlpha is not None:
            currentAlpha = min(2.0, max(1.0, float(targetAlpha)))
        elif randomAlphaOffset > 0:
            currentAlpha = min(2.0, max(1.0,
                                         self.m_baseAlpha + randomAlphaOffset))
        else:
            currentAlpha = self.m_baseAlpha
        gScore = np.full((self.m_height, self.m_width), np.inf)
        gScore[start] = 0
        openList = [(self.Heuristic(start, goal, currentAlpha),
                     0, start)]
        cameFrom = {}
        visited = np.zeros((self.m_height, self.m_width),
                           dtype=bool)
        neighbors = [
            (0, 1, COST_CARDINAL), (0, -1, COST_CARDINAL),
            (1, 0, COST_CARDINAL), (-1, 0, COST_CARDINAL),
            (1, 1, COST_DIAG), (1, -1, COST_DIAG),
```

```

        (-1, 1, COST_DIAG), (-1, -1, COST_DIAG)]
    while openList:
        currentF, currentG, current = \
            heapq.heappop(openList)
        if visited[current]: continue
        visited[current] = True
        if current == goal:
            path = [current]
            while current in cameFrom:
                current = cameFrom[current]
            path.append(current)
            return path[::-1]
        for dr, dc, cost in neighbors:
            nr, nc = current[0]+dr, current[1]+dc
            if not (0<=nr<self.m_height and
                    0<=nc<self.m_width): continue
            if self.m_grid[nr, nc]!=1 or \
                visited[nr, nc]: continue
            newG = currentG + cost
            if newG < gScore[nr, nc]:
                gScore[nr, nc] = newG
                hVal = self.Heuristic((nr, nc),
                                       goal, currentAlpha)
                cameFrom[(nr, nc)] = current
                heapq.heappush(openList,
                               (newG+hVal, newG, (nr, nc)))
    return None

```

A.2 自适应遗传算法核心方法（genetic.py）

以下为自适应遗传算法的核心方法，包括种群初始化、适应度计算、自适应交叉变异率、Softmax 选择算子和进化主循环。

种群初始化与适应度函数：

```

class AdaptiveGA:
    def __init__(self, grid, baseAlpha, start, goal,
                 pop_size=10, max_gen=20, elite_ratio=0.1,
                 chi=1e6, initial_solver="improved_astar"):
        self.m_grid = grid
        self.m_start = start
        self.m_goal = goal
        self.m_popSize = pop_size
        self.m_maxGen = max_gen
        self.m_eliteRatio = elite_ratio
        self.m_chi = chi # 碰撞惩罚系数
        self.m_astar = ImprovedAStar(grid, baseAlpha)
        self.m_pcMax, self.m_pcMin = 0.9, 0.4
        self.m_pmMax, self.m_pmMin = 0.5, 0.1
        self.m_w1, self.m_w2, self.m_w3 = 1.0, 1.0, 1.0

```



```

def InitializePopulation(self):
    self.m_population = []
    attempts = 0
    while len(self.m_population) < self.m_popSize \
        and attempts < self.m_popSize * 10:
        targetAlpha = random.uniform(1.0, 2.0)
        path = self.m_astar.Plan(self.m_start,
            self.m_goal, targetAlpha=targetAlpha)
        if path:
            self.m_population.append(path)
        attempts += 1

def CalculateFitness(self, path):
    if not path or len(path) < 2:
        return -self.m_chi
    L, S = CalculatePathMetrics(path)
    T = len(path)
    P = self.CalculateCollisionPenalty(path)
    cost = self.m_w1*L + self.m_w2*S + self.m_w3*T + P
    return -cost

```

自适应交叉变异率：

```

def GetAdaptiveRates(self, fVal, fAvg, fMax):
    delta = fMax - fAvg
    if delta < 1e-6:
        return self.m_pcMax, self.m_pmMax
    if fVal > fAvg: # 优于平均的个体
        pc = self.m_pcMax - (self.m_pcMax-self.m_pcMin) \
            * (fMax-fVal) / delta
        pm = self.m_pmMin + (self.m_pmMax-self.m_pmMin) \
            * (fMax-fVal) / delta
    else: # 较差个体：高交叉高变异促进改进
        pc = self.m_pcMax
        pm = self.m_pmMax
    return pc, pm

def SoftmaxSelection(self, fitnesses):
    arr = np.array(fitnesses)
    expF = np.exp(arr - np.max(arr))
    probs = expF / np.sum(expF)
    idx = np.random.choice(len(self.m_population),
        p=probs)
    return self.m_population[idx]

```

路径修复机制（A*重连优先）：

```

def RepairPath(self, path):
    repaired = list(path)
    for _ in range(10): # 最多迭代 10 轮修复
        invalidIndices = []
        for i, point in enumerate(repaired):
            y, x = point
            if not (0<=y<self.m_grid.shape[0] and

```

```

        0<=x<self.m_grid.shape[1]):
            invalidIndices.append(i)
    elif self.m_grid[y, x] == 0:
        invalidIndices.append(i)
    if not invalidIndices and \
        self.CheckPathConnectivity(repaired):
        break
    for idx in sorted(invalidIndices, reverse=True):
        if idx == 0 or idx == len(repaired)-1:
            continue
        prev = repaired[max(0, idx-1)]
        nxt = repaired[min(len(repaired)-1, idx+1)]
        # 优先 A*重连
        reconn = self.m_astar.Plan(prev, nxt)
        if reconn and len(reconn) > 2:
            repaired = repaired[:max(0, idx-1)+1] \
                + reconn[1:-1] \
                + repaired[min(len(repaired)-1, idx+1):]
            continue
        # 备选：邻域替代点搜索
        sub = self.FindSubstitutePoint(
            repaired[idx], prev, nxt)
        if sub:
            repaired[idx] = sub
        elif CheckLineOfSight(self.m_grid, prev, nxt):
            repaired.pop(idx)
    return repaired

```

附录 B 实验配置与参数汇总

表 C-1 实验环境配置

配置项	详细信息
操作系统	Windows 11
编程语言	Python 3.12
依赖库	NumPy 1.26, Matplotlib 3.8, Pandas 2.1
实验地图	1500×1500 二维栅格 (map.bytes)
障碍物密度	$\rho = 0.1927$
动态权重	$\alpha(\rho) = 1.3717$

表 C-2 算法参数设置

算法	参数	值
改进 A*	启发式权重 α	1.3717 (自动计算)
改进 A*	启发式函数	曼哈顿距离
改进 A*	邻域模式	8 邻域
自适应 GA	种群大小 N_p	10
自适应 GA	最大进化代数 N_{gen}	20
自适应 GA	精英保留比例 η	0.1
自适应 GA	交叉率范围 P_c	[0.4, 0.9]

自适应 GA	变异率范围 P_m	[0.1, 0.5]
自适应 GA	碰撞惩罚系数 χ	1×10^6
自适应 GA	权重 $\omega_1/\omega_2/\omega_3$	1.0/1.0/1.0
自适应 GA	α 初始化范围	[1.0, 2.0]

表 C-3 测试用例坐标

用例	类型	起点(row, col)	终点(row, col)
1	短距离	(375, 375)	(497, 500)
2	中距离	(300, 301)	(1200, 1200)
3	长距离	(412, 118)	(1382, 1490)